

FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

Quantum Key Distribution Post Processing - A study on the Information Reconciliation Cascade Protocol

André Reis

DISSERTATION



Mestrado Integrado em Engenharia Informática e Computação

Supervisor: José Magalhães Cruz

July 23, 2019

Quantum Key Distribution Post Processing - A study on the Information Reconciliation Cascade Protocol

André Reis

Mestrado Integrado em Engenharia Informática e Computação

Approved in oral examination by the committee:

Chair: Doctor António Miguel Pontes Pimenta Monteiro

External Examiner: Doctor Carlos Filipe Portela

Supervisor: Doctor José Magalhães Cruz

July 23, 2019

Abstract

Quantum Key Distribution (QKD) is a secure key establishment method: it allows two parties to establish a secret key between them. It can be affected by noise, causing the keys held by both parties to be correlated but different. In order to address this issue, there is, usually, a post processing phase that involves an Information Reconciliation (or error correction) step. In this step, one of the parties reveals information about its key and the other uses that information to find and correct the errors in its key.

The amount of information that is disclosed this way is inversely correlated with the number of bits of key that can be securely established using QKD: revealing excessive information about the key increases the advantage of a possible eavesdropper. Conversely, revealing no information is secure but does not allow reconciliation. In fact, there is a theoretical bound for the minimum amount of information needed to be able to reconcile two keys. An optimal protocol would fully correct keys by exchanging the amount of information equal to the theoretical bound. However, there are no such known protocols.

The Cascade protocol is an highly interactive information reconciliation protocol that is currently the standard in QKD because it is of simple implementation, although the amount of information leaked is suboptimal. Multiple studies in current literature have proposed multiple optimizations and modifications to the protocol to make it more efficient. There are also other metrics to analyze in addition to the amount of information leaked, such as the correctness of the protocol (probability to correct all errors) and the number of communication rounds (that affects the throughput). In this dissertation, we perform a study on several Cascade versions proposed in the literature in order to propose a better version.

A practical implementation of multiple Cascade versions was created. It was used to run experiments in order to analyze the evolution of each metric with different key lengths and percentage of errors in the keys. We also propose an optimization to the Cascade protocol, Block Parity Inference, and show it significantly reduces the amount of information leaked for every version. This allows for the proposal of a better Cascade version that uses this optimization.

The proposed Cascade version achieves similar results to the ones in the literature using another optimization, named subblock reuse. A general analysis of both optimizations indicates that an integration of both optimizations would be an even larger improvement on the efficiency of Cascade, having better results than any previous study on the Cascade protocol.

Resumo

Quantum Key Distribution (QKD) é um método seguro de estabelecimento de chaves: permite que duas entidades estabeleçam uma chave secreta entre elas. Este pode ser afetado por ruído, fazendo com que as chaves obtidas por ambas as entidades sejam correlacionadas, mas diferentes. Para resolver este problema, geralmente há uma fase de pós-processamento que envolve uma etapa de Reconciliação de Informação (ou correção de erros). Nesta etapa, uma das entidades revela informação sobre sua chave e a outra usa essa informação para encontrar e corrigir os erros na sua chave.

A quantidade de informação que é divulgada desta forma é inversamente correlacionada com o número de bits de chave que podem ser estabelecidos com segurança usando QKD: revelar informação excessiva sobre a chave aumenta a vantagem de um possível adversário que observe a comunicação. Por outro lado, não revelar nenhuma informação é seguro, mas não permite a reconciliação. De facto, há um limite teórico para a quantidade mínima de informação necessária para poder reconciliar duas chaves. Um protocolo ideal corrigiria completamente as chaves, trocando uma quantidade de informação igual ao limite teórico, no entanto, não se conhecem protocolos ideais.

O protocolo Cascade é um protocolo de Reconciliação de Informação altamente iterativo que atualmente é convencional em QKD por ser de simples implementação, embora a quantidade de informação revelada não seja ideal. Vários estudos, em literatura atual, propuseram múltiplas otimizações e modificações ao protocolo para o tornar mais eficiente. Há também outras métricas para analisar, além da quantidade de informação revelada, a correção do protocolo (probabilidade de corrigir todos os erros) e o número de mensagens trocadas (que afeta a velocidade de execução). Nesta dissertação, realizamos um estudo sobre várias versões de Cascade propostas na literatura, a fim de propor uma versão ótima.

Desenvolveu-se uma implementação prática de várias versões de Cascade. Esta foi usada para executar experiências com o objetivo de analisar a evolução de cada métrica com diferentes comprimentos de chave e percentagem de erros nas chaves. Também propomos uma otimização para o protocolo Cascade, Block Parity Inference (inferência da paridade de blocos) e mostramos que isso reduz significativamente a quantidade de informação revelada para cada versão. Isto permite a proposta de uma versão de Cascade ideal que usa esta otimização.

A versão de Cascade proposta alcança resultados semelhantes aos obtidos na literatura usando outra otimização, chamada *subblock reuse* (reutilização de subblocos). Uma análise geral a ambas as otimizações indica que uma integração destas provocaria uma melhoria ainda maior na eficiência do protocolo, tendo melhores resultados do que qualquer estudo anterior ao protocolo Cascade.

Acknowledgements

First, I would like to thank Professor David Elkouss Coronas for helping me find an interesting topic in this area, create a draft proposal and for setting me on the right track in this project multiple times.

I would also like to thank my supervisor, Professor José Magalhães Cruz, for accepting the challenge of helping me in this thesis of previously uncharted territory.

I would like to thank the staff of my host institution, Fractal Blockchain, specially my supervisor, Hugo Peixoto, for taking me in so kindly and helping me in a daily basis by either showing me interesting concepts I didn't know about and by finding bugs in my code.

Last but not the least, I would like to thank everyone that has helped me in any way during this project and in my academic years: my family, my friends... I wouldn't be able to achieve this all by myself.

André Reis

“God does not play dice with the universe; He plays an ineffable game of His own devising, which might be compared, from the perspective of any of the other players [i.e. everybody], to being involved in an obscure and complex variant of poker in a pitch-dark room, with blank cards, for infinite stakes, with a Dealer who won’t tell you the rules, and who smiles all the time.”

Terry Pratchett

Contents

1	Introduction	1
1.1	Motivation and Context	1
1.2	Objectives	2
1.3	Document Structure	2
2	Background	5
2.1	Information Theory	5
2.1.1	History and Basic Concepts	5
2.1.2	Error correction codes	6
2.2	Quantum Key Distribution	7
2.3	Conclusion	9
3	Literature Review	11
3.1	Cascade	11
3.1.1	Original	15
3.1.2	Cascade with BICONF	15
3.1.3	BICONF as Cascade	17
3.1.4	Full optimization analysis	17
3.2	Conclusion	18
4	Studying the Information Reconciliation Cascade Protocol	19
4.1	Problem Description	19
4.2	Implementation	20
4.2.1	Dataset Generator	21
4.2.2	Algorithm Executor	21
4.2.3	Algorithm Implementation	22
4.2.4	Run Validation	23
4.2.5	Results Processing	24
4.2.6	Other Command Line options	25
4.3	Block Parity Inference	25
4.3.1	Description	25
4.3.2	Implementation	25
4.4	Conclusion	27
5	Experiments and Results	29
5.1	Experiments	29
5.1.1	First experiment results	30
5.1.2	Second experiment results	33

CONTENTS

5.2	Conclusions	35
6	Conclusions and Future Work	37
6.1	Conclusions	37
6.2	Future Work	38
A	Appendix 1	41

List of Figures

2.1	Syndrome coding	6
2.2	Interactive error correction	7
2.3	Quantum Key Distribution	8
3.1	Binary protocol	14
4.1	Evolution of the channel uses array during the first iteration.	23
4.2	Evolution of the channel uses array during the first iteration.	26
5.1	First experiment reconciliation efficiency by error rate on keys with 10000 bits . .	30
5.2	First experiment frame error rate by error rate on keys with 10000 bits	31
5.3	First experiment number of channel uses by error rate on keys with 10000 bits . .	31
5.4	Reconciliation efficiency by error rate for keys with 1024 and 2048 bits	32
5.5	Reconciliation efficiency and channel uses by key length	32
5.6	Reconciliation efficiency by error rate on keys with 16384 bits	33
5.7	Reconciliation efficiency by key length with 5% error rate and 6% error rate . . .	33
5.8	Reconciliation efficiency of the first experiment (on the left) and second (on the right)	34
5.9	Channel uses of the first experiment (on the left) and second (on the right)	34
5.10	Frame error rate of the first experiment (on the left) and second (on the right) . .	34
5.11	Reconciliation efficiency by error rate, from [1].	35

LIST OF FIGURES

List of Tables

3.1	Strings split in blocks and corresponding parities	14
3.2	Strings split in blocks and corresponding parities after first iteration	15
3.3	Strings split in blocks and corresponding parities in the second iteration	15
3.4	First iteration state after second iteration correction	15
3.5	Cascade versions from [1], adapted.	18
5.1	Analyzed Cascade versions in both experiments	30
A.1	First Experiment Results	42
A.2	Second Experiment Results	68

LIST OF TABLES

Abbreviations and Symbols

AVG	Average
BER	Bit Error Rate
BPI	Block Parity Inference
BSC	Binary Symmetric Channel
CLI	Command Line Interface
CSV	Comma-Separated Values
CU	(Number of) Channel Uses
EFF	Efficiency
FER	Frame Error Rate
QKD	Quantum Key Distribution
VAR	Variance
C_A, C_B	Symbols used to represent syndromes of A and B
f_{EC}	Reconciliation efficiency
$H(\cdot)$	Shannon's entropy
H	Symbol used for a parity check matrix
h	Binary entropy
k_i	Symbol for the size of the block for iteration i
m, n	Symbols used for dimensions
p	Symbol used for probability
p_i	Symbol used for the probability of i
Q	Symbol used for the error rate (from Quantum BER)
x_A, x_B, A, B	Symbols used for strings of A and B
ϵ	Symbol used for dimensions associated with errors

Chapter 1

Introduction

1.1 Motivation and Context

Quantum Key Distribution (QKD) is a secure key generation method: it allows two parties to establish a secret key (to communicate using symmetric encryption and/or message authentication codes) between them [2]. The protocol should be provably secure, based on the laws of Quantum Mechanics, against an adversary with unbounded computational power. However, it can be affected by noise of the channel or disturbance caused by an attacker which causes the keys shared by both parties to be correlated but different. Hence, there should be a post-processing step which includes information reconciliation (or error correction).

In order to have a reliable secret key agreement, the correctness of the used information reconciliation protocol is very important, that is, the protocol should be able to correct all discrepancies in the secret key. In order for the protocol to be secure, the maximum possible size of the key established using QKD is decreased by the number of bits possibly revealed during the Information Reconciliation step. Thus, we are presented with an optimization problem: maximize the correctness of the protocol and minimize the information leaked by the necessary communication, that is public.

Gilles Brassard and Louis Salvail proposed an information reconciliation protocol, Cascade [3] which is currently *"the de-facto standard for practical implementations of information reconciliation in quantum key distribution"* [1]. Cascade is a highly interactive protocol that has customizable parameters that we will study in order to achieve the best optimization possible. The protocol is, in general, more efficient the bigger the keys are which is good for QKD since it allows establishing a key big enough to use with one-time pad encryption [4, 5] or to keep leftover key bits for future interactions¹ of the same pair of parties.

¹Specially important, those bits can be used to generate Message Authentication Codes for future QKD executions to extend the length of the shared key

The work in this dissertation is aligned with the company Fractal’s research efforts in the blockchain space and the future of its technology after the rise of quantum computing. The company, within which the dissertation’s work was conducted, builds web applications and services for worldwide identity verification, in order to enable inclusive access to global financial markets. Fractal is currently focused on the blockchain fintech world, as its players are of a new breed which think globally instead of in national silos. Fractal observes the inevitable modularization of the global financial stack, and its identity solution is one of the crucial components of this stack.

1.2 Objectives

Different versions of the Cascade protocol have their advantages and disadvantages. These are related to the optimization parameters relative to channel noise, such as correctness and the amount of information leaked.

The main objective of this dissertation is to analyze different versions of the Cascade protocol in order to propose an optimized version.

In order to perform this analysis, there was a need to create an application capable of generating datasets of key pairs with errors (given the key length and error rate), running a given algorithm for a given dataset (and outputting the statistics for each run) and replicating a run, to provide both replicability and reproducibility. After the design of this application and the implementation of all algorithms, experiments were ran using generated datasets with different key lengths and error rates.

We propose (and implement) an optimization to the Cascade protocol, Block Parity Inference. By running additional experiments using the optimization, we show that it reduces the amount of information exchanged and the number of channel uses by trading off memory and processing power. We propose that all exchanged block parities are kept in memory and before any parity request, the memory is searched through to look for a combination of parities that could allow the inference of the desired parity.

Given the results of these experiments, we will propose a Cascade version as the most optimal. In addition to this, we contribute with a software that facilitates further studies, since it is of simple extension. It is very straightforward to create new Cascade versions to perform other experiments. We hope to have built a tool for community use.

1.3 Document Structure

This document has five more chapters.

Chapter 2 briefly summarizes the most important concepts needed for the understanding of the work done in this dissertation: error correction codes and Quantum Key Distribution.

Chapter 3 contains a literature review of the current status of the main topics of this dissertation: error correction codes and Cascade protocol.

Introduction

Chapter 4 describes the problem, the details of the implementation of the developed software and proposed optimization.

Chapter 5 explains the experiments performed, presents and discusses the results obtained.

Finally, Chapter 6 concludes this dissertation by remembering its contributions and an analysis of possible future work.

Introduction

Chapter 2

Background

This chapter will present the basic concepts required to understand this dissertation. Starting with the a brief explanation of error correction codes, in the field of Information Theory, the necessary basics of Quantum Key Distribution will follow.

2.1 Information Theory

2.1.1 History and Basic Concepts

The early days of the Information Theory field were marked by Claude Shannon who wrote "A Mathematical Theory of Communication" [6]. This field studies information and the fundamental limits for its quantification and communication.

During the following years, the field had great developments that were responsible for many technologies used nowadays such as: lossless data compression of files (e.g: ZIP), lossy data compression for lightweight filetypes like JPEG and channel coding for digital communication over telephone lines. Another development was the concept of entropy (usual notation $H(\cdot)$) which is the uncertainty contained in the value of a random variable, essential for the development of cryptography. For a random variable X that takes values x_i with respective probabilities p_i , the entropy of X , $H(X)$ is given by the following formula:

$$H(X) = - \sum_i p_i * \log_2(p_i)$$

.

Some basic Information Theory concepts related to this dissertation follow:

- Channel (or communication channel) is a transmission medium for communication, e.g., a fiber optic cable connecting two computers.

Background

- Channel capacity is the maximum rate at which information can, reliably, be transmitted over a channel.
- A noisy channel is a channel that, with some probability, transmits information with errors. There are different channel models for noisy channels that characterize the error and its probability; we will assume the communication is made over a binary symmetric channel.
- A binary symmetric channel (BSC) is a model of noisy channel where a bit is received correctly with probability $1 - p$ and the opposite value with probability p .
- Binary entropy, $h(p)$, is the uncertainty contained in the value of a random variable that can only take two values with probabilities p and $1 - p$ (this is called a Bernoulli Trial). It can be calculated by using the formula of $H(X)$: $h(p) = -p * \log_2(p) - (1 - p) * \log_2(1 - p)$. As an example, the entropy of a bitstring, A , of length n where each bit is chosen as the result of a Bernoulli trial with probability 0.5 is $H(A) = n$.
- Conditional entropy, $H(A|B)$, is the uncertainty contained in the value of a random variable, A , having knowledge of another variable B (A and B can be bit strings). Intuitively, if the variables are completely independent, $H(A|B) = H(A)$ (knowing B gives no additional information about A); if they are completely dependent $H(A|B) = 0$ (knowing B means we know A , there is no uncertainty).

2.1.2 Error correction codes

To deal with noisy channels, Information Theory studies error correction codes which are protocols that communicate redundant information, e.g. bit parity data, so that it will be possible to find and correct errors in the bit strings received. There are several types of error correction codes but the most relevant for this dissertation are "syndrome coding" and "interactive error correction".

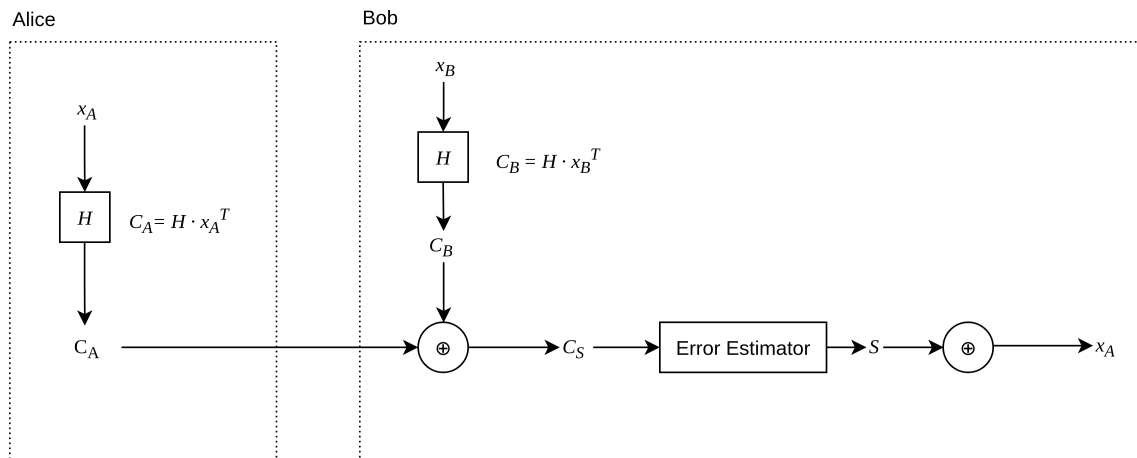


Figure 2.1: Syndrome coding

Background

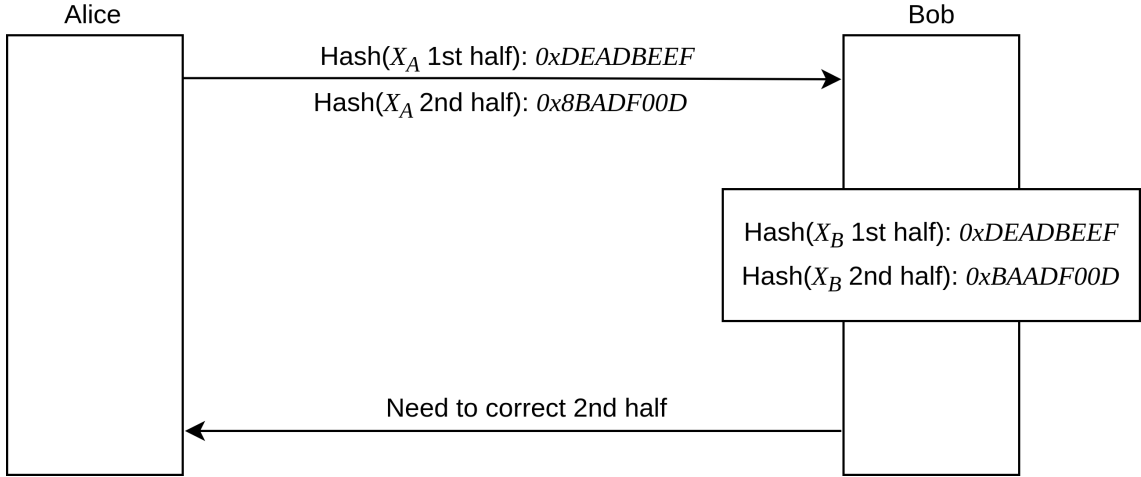


Figure 2.2: Interactive error correction

Syndrome coding, as seen in Fig 2.1, is a construction that uses a $m \times n$ parity matrix H^1 , where m is the length of the strings to correct and n the chosen length for the syndrome, that is agreed upon by both parties. Given a string x_A of length m , the multiplication $H \cdot x_A'^2$ is called its syndrome C_A . An example of an error correction protocol using syndrome coding is: Alice sends her syndrome C_A to Bob, who also calculates his syndrome C_B . Bob then calculates $C_S = C_A \oplus C_B$, which is usually called the error syndrome, because it is the syndrome of the error string. Then C_S is sent to a module that estimates the error string S (e.g: from C_S find S such that $H \cdot S^t = C_S$) and then Bob's string X_B is corrected by calculating $X_B \oplus S$.

Interactive protocols, as seen in Fig 2.2, involve more steps and two-way communication. The intuition about these codes is that Bob asks questions about properties (e.g. the parity or the cryptographic hash value) of parts of Alice's string and compares her answer with the data from his received string, and enters a correction protocol if it is different.

Based in Shannon's Noiseless Coding Theorem presented in [6], a particular case of Slepian-Wolf coding (source coding with side information), presented in [7], establishes a lower bound on the required amount of information transmitted in order to achieve correct reconciliation. This lower bound is the conditional entropy in the strings, $H(A|B)$. For a BSC with error probability p , $H(A|B) = nh(p)$, where n is the length of the string to be reconciled and $h(p)$ the binary entropy of p .

2.2 Quantum Key Distribution

Quantum Key Distribution [8, 9] is an emerging secret key generation method that uses quantum technology. It was introduced by Charles Bennett and Gilles Brassard [2], called the BB84 protocol; later, a slightly different approach was taken by Arthur Ekert [10], originating the E91

¹Not to confuse with the Shannon's entropy, $H(\cdot)$

² x_A' is the transpose of the string x_A : x_A is seen as a row vector. To be able to perform the multiplication it is transposed to a column vector.

Background

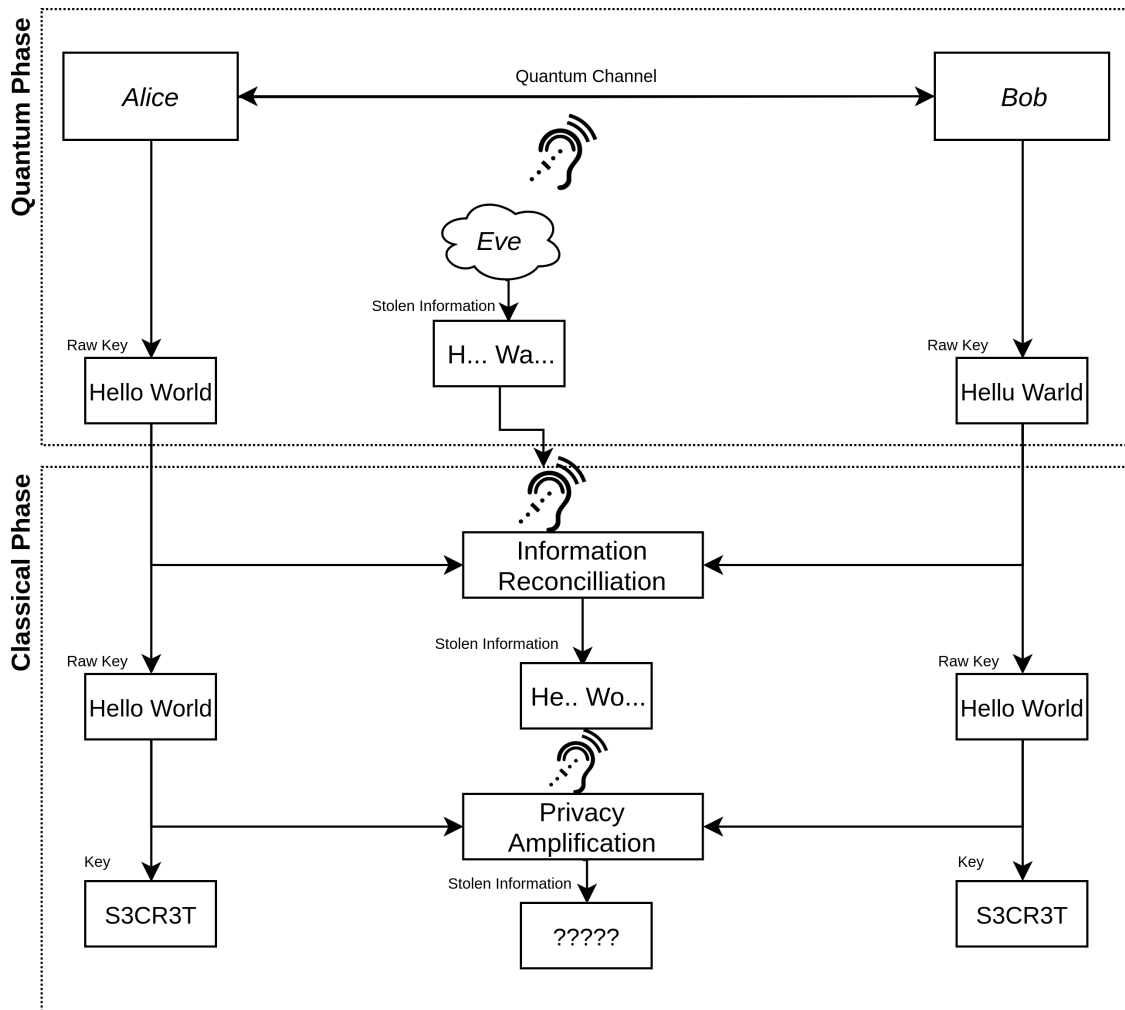


Figure 2.3: Quantum Key Distribution

protocol. The advantage of these protocols in comparison with the Diffie-Hellman protocol [11] for example, is that their security does not rely on any mathematical hard problem but on the laws of Quantum Mechanics.

QKD has two main phases: the quantum phase and the classical phase (or post processing)[12], which are shown in in Fig.2.3. The quantum phase uses a public quantum communication channel and the classical phase uses a public classical authenticated channel (using Digital Signatures or Message Authentication Codes). During the quantum phase, the parties use quantum technology to establish a secret key with a length which was previously agreed upon. The quantum communication channel is usually noisy, so the key establishment is not perfect and there will be differences in the keys obtained by both parties. As said, this communication is public, so there is the possibility that an eavesdropper obtains information about the key; however, the amount of information that can be obtained without the snooping being noticed is limited because an attempt to obtain information introduces more noise.

This is addressed in the classical phase: in an initial step, both parties reveal a number of bits³ of their established key in order to estimate the percentage of errors (error rate) between their keys. If the error rate is too high it is not possible to securely establish a key and the protocol is aborted (successfully avoiding any possible attack⁴). In the other case, both parties use the error rate to estimate the possible amount of information obtained by an eavesdropper and then proceed to apply an error correction protocol. As such, they will obtain identical keys, in what is called the Information Reconciliation step. As this step uses communication over a public channel, an eavesdropper can learn information about a number of bits, c , where c depends of the efficiency of the algorithm for the given key length and error rate. After this, assuming the previous step was successful, they perform Privacy Amplification in order to minimize the eavesdropper's knowledge about the key.

Privacy Amplification extracts a key of n bits from a raw key of m bits ($m > n$) that will look uniformly random to an eavesdropper as long as they do not have more than $n - 1$ bits of information about the raw key. This comes from the Leftover Hash Lemma [13]. Usually, a security parameter ε is included in the formula (and affects it by requiring the existence of $\log_2(\frac{1}{\varepsilon})$ additional bits) to ensure this process is secure even in the worst case scenario. As previously mentioned, the eavesdropper can also obtain c bits of information about the key in the Information Reconciliation step, therefore, in order to generate n bits of key that look random to the eavesdropper, the eavesdropper should only have acquired less than $n - 1 - c - 2 * \log_2(\frac{1}{\varepsilon})$ bits of knowledge about the raw key [13].

2.3 Conclusion

This chapter presented the main concepts of error correction codes and Quantum Key Distribution, detailing the importance of the efficiency of the error correction algorithm for the security of QKD. The next chapter will review current literature on error correction codes and, more specifically, the Cascade protocol and its modifications.

³These bits are then discarded because they are public, and therefore, not useful for secret key establishment.

⁴To be precise, a Denial of Service (DoS) attack is possible by actively eavesdropping and/or introducing noise. However, it is not relevant since classical protocols are affected in the same way by this kind of attack.

Background

Chapter 3

Literature Review

This chapter presents a summary over the literature related to the main topic of this dissertation. It focuses on the Cascade Protocol and the proposals for modifications and optimizations.

3.1 Cascade

Brassard and Savail present the Information Reconciliation problem and how it can be optimally solved [3]. They also show that it is hard to generate optimal reconciliation protocols (in fact, they showed there are no known efficient algorithms), so they present the idea of almost-ideal protocols. They present Cascade as a protocol that reveals an amount of information close to the theoretical bound. The Cascade protocol uses the Binary protocol proposed in *Experimental quantum cryptography*[14], that works as follows:

1. «Alice sends Bob the parity of the first half of the string
2. Bob determines whether an odd number of errors occurred in the first half or in the second by testing the parity of the first half and comparing it to the parity sent by Alice.
3. This process is repeatedly applied to the half determined in step 2. An error will be found eventually.»[3]

The whole process is presented in pseudocode in Algorithm 1, having as input the block (or string) with an odd number of errors and returning the index of the block that contains an error. It is important to note that the *askBlockParity* function involves communication to ask the parity of the given block and outputs the received parity, while the *calculateParity* function computes the XOR between the bits of given block.

Algorithm 1: Binary

```

Input: Block
Result: ErrorIndex
if Block.length = 1 then
    | return Block.getIndex();
else
    | firstHalf := Block.getSubBlock(0, Block.length / 2);
    | correctFirstHalfParity := askBlockParity(firstHalf);    // Remote function call
    | currentFirstHalfParity := calculateParity(firstHalf);
    | if correctFirstHalfParity ≠ currentFirstHalfParity then
    | | return Binary(firstHalf);
    | else
    | | secondHalf := Block.getSubBlock(Block.length / 2, Block.length);
    | | return Binary(secondHalf);
    | end
end

```

The Cascade protocol works as follows:

Iteration 1:

1. Alice and Bob agree on the number of bits for the block size, k_1
2. Alice and Bob split their strings in continuous blocks of size k_1
3. Alice sends the parities of her blocks to Bob
4. For each block where the parities are not equal, Bob fixes one error using the Binary protocol

Iteration n (for $n > 1$):

1. Alice and Bob agree on the number of bits for the block size, k_n
2. Alice *shuffles* her string: she creates blocks with given block size but instead of continuous blocks, she chooses k_n positions on the string to build a block and sends this information to Bob
3. Repeat first iteration steps 3-4

At this point it is important to notice that upon correcting an error (in a bit with index i) in this step, they uncover that for any previous iteration, its block that contained the same index must have had an even number of errors, and with the bit with index i corrected, that block now has an odd number of errors.

4. Given this, for each block where they correct a bit, they create a set with the blocks from previous iterations that contained that bit.
5. They use the binary protocol to correct one error in each block in the set, starting with the smallest blocks.

However, each of these corrections will cause the same effect described at the end of step 3. So, for each correction (in a bit with index i) they add to the set of blocks to correct the corresponding block from every iteration until n (inclusively) except they should not add the block they just corrected. This step is the reason for the name of the protocol, since each correct will trigger a *Cascade* of corrections. This is usually referred to as trace-back step or cascade effect.

The Cascade protocol is represented in Algorithm 2, it receives as input the raw key to correct and the number of iterations to execute, returning the key, corrected. The previously described Cascade Effect or trace-back was split into its own function in Algorithm 3 for clarity, it receives from the Cascade protocol the raw key, the number of the iteration being processed and the index where an error was found and performs corrections on the received raw key.

Algorithm 2: Cascade

Input: RawKey, NumIterations

Output: CorrectedKey

```

for iterationNumber  $\leftarrow$  0 to NumIterations do
    iterationBlocks := getIterationBlocks(RawKey, iterationNumber);
    currentBlockParities := calculateParities(iterationBlocks);
    correctBlockParities := askParities(iterationBlocks); // Remote function call
    for blockNumber  $\leftarrow$  0 to currentBlockParities.length do
        if correctBlockParities[blockNumber]  $\neq$  currentBlockParities[blockNumber] then
            errorIndex := Binary(iterationBlocks[blockNumber]);
            RawKey[errorIndex] :=  $\neg$  RawKey[errorIndex];
            cascadeEffect(RawKey, iterationNumber, errorIndex);
        end
    end
end
return RawKey;

```

Algorithm 3: CascadeEffect

Input: RawKey, LastIteration, FirstErrorIndex

setOfErrorBlocks := PriorityQueue(order by length: crescent);

currentIteration := LastIteration;

currentErrorIndex := FirstErrorIndex;

```

do
    for iterationNumber  $\leftarrow$  0 to LastIteration + 1 do
        if iterationNumber  $\neq$  currentIteration then
            block := getCorrespondingBlock(iterationNumber, currentErrorIndex);
            setOfErrorBlocks.append(block);
        end
    end
    errorBlock := setOfErrorBlocks.pop();
    if getParity(errorBlock)  $\neq$  getCorrectParity(errorBlock) then
        currentIteration := errorBlock.iteration;
        currentErrorIndex := Binary(errorBlock);
        RawKey[errorIndex] :=  $\neg$  RawKey[errorIndex];
    end
while setOfErrorBlocks is not empty;

```

We will now illustrate with an example. The initial strings and parities are as seen in table 3.1 (the example has $k_1 = 4$). There is only one block where the parity is different; they start Binary for the first block. The Binary protocol will go as shown in Fig. 3.1.

They will execute the Binary protocol for the first block and correct the error on the first bit of the string. Then they will go to the previous iteration and see the state represented in table 3.4. They will see that Bob's parity for the first block is now different from Alice's and perform Binary on the first block, fixing the error on the second bit of the string.

Table 3.1: Strings split in blocks and corresponding parities

Alice's Blocks	1	1	1	1
Bob's Blocks	0	0	0	1
Alice's Parities	0			
Bob's Parities	1			

Table 3.2: Strings split in blocks and corresponding parities after first iteration

Alice's Blocks	1 1 1 1	1 1 1 1	1 1 1 1	1 1 1 1
Bob's Blocks	0 0 1 1	1 1 1 1	1 1 1 1	1 1 1 1
Alice's Parities	0	0	0	0
Bob's Parities	0	0	0	0

Table 3.3: Strings split in blocks and corresponding parities in the second iteration

Alice's Blocks	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
Bob's Blocks	0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1
Alice's Parities	0
Bob's Parities	1

Table 3.4: First iteration state after second iteration correction

Alice's Blocks	1 1 1 1	1 1 1 1	1 1 1 1	1 1 1 1
Bob's Blocks	1 0 1 1	1 1 1 1	1 1 1 1	1 1 1 1
Alice's Parities	0	0	0	0
Bob's Parities	1	0	0	0

3.1.1 Original

The original version of Cascade, proposed by Brassard and Savail[3], has the following parameters: 4 iterations, $k_1 = \lceil 0.73/Q^1 \rceil$, $k_i = 2k_{i-1}$. The choice of block sizes is such that the probability of a block containing an error decreases exponentially with the number of iterations (proof in the referred document). The choice of the number of iterations was made with 10 empirical tests (for strings of 10000 bits) that also showed that the average amount of leaked information was close to the theoretical bound.

However, following studies (e.g. [1, 15, 16, 17]) have proved that the amount of information leaked during the protocol is suboptimal and that it is possible to improve the efficiency of the Cascade protocol by tuning the protocol's parameters: number of iterations and block size for each iteration. There have also been proposals to improve the efficiency of Cascade by either modifying the protocol or by optimizing some processes [1, 15]. The proposals relevant for this study will be reviewed in the following sections.

3.1.2 Cascade with BICONF

In Sugimoto et al.[16], a modification to the Cascade protocol is proposed. It was noticed in 100 empirical tests (for strings of 10000 bits) that after the second iteration of Cascade almost all the errors were corrected. In fact, almost half of the errors were corrected in the first iteration and the other half in the second, so they argue that it is not worthwhile to have more than two cascade

¹Q is the estimated error rate in the strings

iterations. However, there probably still are a few errors in the strings after the second cascade iteration so the protocol should not simply terminate after it. In order to correct these errors, they propose that another protocol, BICONF^s is executed after the second iteration. This protocol works as follows:

1. Alice and Bob choose a random subset of corresponding bits from their strings
2. Alice sends Bob the parity of her subset
3. If the parity of Bob's subset is different, they execute BINARY once for the chosen subset and another for the complementary subset
4. They repeat steps 1-3 until Bob finds no errors in s successive repetitions.

The biconf protocol is presented in pseudocode in Algorithm 4, having as input the raw key and the number of biconf iterations (previously referred to as s) and returning the corrected raw key. In the same paper, they estimate that the probability that this protocol fails to correct all errors is less than 2^{-s} and choose $s = 10$. They also argue that the block sizes: $k_1 = \lfloor \frac{4 \ln 2}{3Q} \rfloor$, $k_2 = 3k_1$ are more adequate to this modification of the Cascade protocol. No method of choosing the random subset for the BICONF protocol is presented, and in following experiments, such as the ones conducted in Martinez-Mateo et al.[1], a Bernoulli trial with probability 0.5 is used to decide, for each bit, if it is part of the subset.

Algorithm 4: Biconf

Input: RawKey, NumBiconfIterations

Output: CorrectedKey

iterationNumber := 0;

while iterationNumber < NumBiconfIterations **do**

iterationBlocks := splitInTwoBlocks(RawKey);

currentFirstBlockParity := calculateParity(iterationBlocks[0]);

correctFirstBlockParity := askParity(iterationBlocks[0]); // Remote function call

if currentFirstBlockParity $\neq$ correctFirstBlockParity **then**

errorIndex := Binary(iterationBlocks[0]);

RawKey[errorIndex] := $\neg$ RawKey[errorIndex];

errorIndex := Binary(iterationBlocks[1]);

RawKey[errorIndex] := $\neg$ RawKey[errorIndex];

end

end

return RawKey;

3.1.3 BICONF as Cascade

In Yan et al.[15], the authors notice that one BICONF iteration is almost equivalent to a Cascade iteration with block size $N/2$, but it can even correct more errors due to triggering the cascade effect upon correcting an error. The authors also perform an experiment of 100 empirical tests (for strings of 10000 bits) to determine the best block sizes for the first two iterations so that they can correct the maximum possible amount of errors in these two iterations. After the second iteration, in order to correct the few remaining errors, they propose 8 more iterations with block size $N/2$. The proposed protocol has: 10 iterations, $k_1 = 0.8$, $k_2 = 5k_1$ and $k_i = \lceil N/2 \rceil (i > 2)$.

Although no practical implementation or experiment was made, in the same paper, the authors introduce the concept of Block Reuse². They argue that it is possible to optimize the protocol by maintaining a record of all subblocks exchanged during BINARY executions and also using them for the cascade effect. That is, after the first iteration, upon correcting one error, instead of adding to the set of blocks to correct just the block from each iteration containing the corrected index, every subblock containing that index that was exchanged during the BINARY protocol should also be added to the set. As these blocks will be smaller, fewer parity exchanges will be needed to correct an error.

3.1.4 Full optimization analysis

The previous papers proposing the original cascade protocol [3] and both modifications mentioned [15, 16] so far are very limited: they focus only on reconciliation efficiency and they perform analysis over 100 tests of the algorithm just for one key length: 10000 bits.

In Martinez-Mateo et al.[1] a more thorough analysis is made, focusing on main parameters: Reconciliation Efficiency, Robustness (Frame Error Rate³ and Bit Error Rate⁴) and Number of Channel Uses (or communication rounds). Using these parameters for analysis gives a better idea of the trade-offs offered by each modification. This study works with datasets of 10^6 algorithm runs so it can have high precision for Frame Error Rate and Bit Error and even though most of the analysis was made for key lengths of 10^4 bits it is not limited to it and the final proposal for the optimized protocol uses a key length of 2^{14} (16384) bits.

For the analysis of the number of uses of the communication channel, the protocol is parallelized: "[...] blocks and parities are processed in parallel. Therefore, instead of exchanging messages with single parities typically a set of parities (i.e., a syndrome) are processed and communicated. In what follows, all the non-dependent information is collected in one message until the protocol can no longer proceed and the message is transmitted. Our results show then the minimal number of messages needed. Note that dichotomic searches (i.e., subblock parities) are also processed in parallel." [1]. However, no practical implementation is shown and there is no clear reasoning for the parallelization in the cascade effect, since there is a trade-off of parallelization vs. efficiency: if all elements in the priority queue at each iteration are run in parallel it is possible

²Or Subblock Reuse

³Probability that the protocol fails

⁴Probability of a bit having the wrong value when the protocol fails

Table 3.5: Cascade versions from [1], adapted.

Protocol	Block Sizes (approx.)			Cascade passes	BICONF	Block reuse
	k_1	k_2	k_i			
original Ref.[3]	$0.73/Q$	$2k_1$	$2k_{i-1}$	4	no	no
biconf Ref.[16]	$0.92/Q$	$3k_1$	-	2	yes	no
yan et al. Ref.[15]	$0.8/Q$	$5k_1$	$n/2$	10	no	no
option-7 Ref.[1]	$2^{\lceil \log_2 1/Q \rceil}$	$4k_1$	$n/2$	14	no	yes
option-8 Ref.[1]	$2^{\lceil \alpha \rceil}$	$2^{\lceil (\alpha+12)/2 \rceil}$	$k_3 = 2^{14} = 4096$ $k_i = n/2$	14	no	yes

$$\alpha = \log_2(1/Q) - \frac{1}{2}$$

The names of the protocols were adapted for clarity. However, as two protocols were proposed in Martinez-Mateo et al.[1] and have no defining features, the original name was kept (option-7 and option-8)

that more parities are exchanged due to executing binary in larger blocks than the ones that would be executed if the process is not parallelized (it would run binary in the smallest block and then more blocks would be added to the priority queue and so on).

In the same paper, the authors analyze the Cascade versions mentioned so far and present 5 more own proposals. The relevant Cascade versions for this dissertation are listed in Table 3.5. As a result of the study, they propose option-8 as the most optimized version.

3.2 Conclusion

In this chapter, the current literature [1, 3, 15, 16, 17] on Cascade and proposals of modifications for its improvement has been reviewed. Except for Martinez-Mateo et al.[1], the experiments in the literature are very limited both in the number of tests and in the parameters that are analyzed. Also, there are no practical implementations or code available in order to validate the results.

In the next chapters we will address this issue by presenting our implementation and the performed experiments.

Chapter 4

Studying the Information Reconciliation Cascade Protocol

This chapter presents the problem description, explaining the variables of the optimization problem. This is followed by a detailed description of the tool developed to study the problem.

4.1 Problem Description

As previously stated, in a Quantum Key Distribution system, it is of high importance that the Information Reconciliation protocol leaks the minimum amount of information possible. The ratio between the total amount of bits exchanged, m , and theoretical minimum established in [7] is called the Reconciliation Efficiency, f_{EC} . For strings A and B of length n , assuming the channel is a BSC with error probability ε , f_{EC} is given by:

$$f_{EC} = \frac{m}{H(A|B)} = \frac{m}{nh(\varepsilon)}$$

As $H(A|B) = nh(\varepsilon)$ is the theoretical minimum of information exchanged to reconcile the strings A and B , we have that $f_{EC} \geq 1$ and the protocol is optimal for $f_{EC} = 1$ (this is also referred to as perfect reconciliation).

The reliability (or robustness) of the protocol is, naturally, inversely correlated to its probability of failure. In order to evaluate this characteristic of the protocol the Frame Error Rate (FER) is used. This is the probability that the protocol fails to reconcile all errors, that is, there is at least one difference between the strings, or, in the end of the protocol $A \neq B$. This metric is complemented by the Bit Error Rate (BER), which is the ratio between the number of differences in the strings (given by the Hamming Distance between the strings) and their length.

$$FER = Pr(A \neq B), BER = \frac{HammingDistance(A,B)}{n}$$

As Cascade is a highly interactive protocol, high latencies can cause it to have very high execution time. For this reason, it is essential to minimize the number of channel uses (or communication rounds) so that the Information Reconciliation step is not a bottleneck in the key exchange process. The number of channel uses is the number of messages exchanged in the communication channel. We will take an approach like the one previously mentioned in [1] and simulate parallelization in all non-dependent information, exchanging messages with sets of parities instead of single parities as often as possible.

Formally, we are posed with an optimization problem that aims to minimize the Reconciliation Efficiency, the Frame Error Rate, Bit Error Rate and Number of Channel uses. The protocol should also be adaptive given the key length and error rate: it is possible that QKD is performed using communication channels affected by different amounts of noise or require different key lengths.

For these reasons, the solution will be a set of parameters for Cascade (and possibly protocol modifications) that provide the best optimization over the defined parameters. It is possible that there is no version of the protocol that is optimal over all defined optimization parameters, and the series of trade-offs should be presented.

4.2 Implementation

In this section, we will describe the implementation of the software that facilitated this study. Besides implementing the algorithms in Table 3.5 and being able to retrieve the relevant statistics, the developed tool should also allow the validation of results, in order to ensure replicability and reproducibility. For convenience, functionalities for the processing of the retrieved statistics were also created. These features were used in the following workflow: generate datasets; run all algorithms for each dataset; process the results and retrieve the statistics.

The program was developed in Python because of the versatility of the language, the provided libraries, as well as the global reach it provides: it should be simple to use this tool as a reference implementation of Cascade or to extend it for other purposes. The source code and documentation for this tool are publicly available¹.

A command line interface (CLI) was designed to provide access to the functionalities using the following syntax:

```
$ cascade-study <command> <parameters>*
```

The existing commands are: *create_dataset*; *run_algorithm*; *validate_run*; *process_results*; *create_chart*. Their detailed description will follow.

¹In thesis.andreireis.eu/source-code

4.2.1 Dataset Generator

In order to separate the results from the input data and to reduce the processing done on each algorithm run, a command to generate datasets of key pairs for a given key length and error rate was designed. The syntax for the command is:

```
$ cascade-study create_dataset <key length> <error rate> <options>*
```

Between the utility options, the number of key pairs in the dataset can be defined, otherwise the default 10^5 is used.

In order to simulate the noisy channel behaviour as a BSC (with the error rate as the probability of flipping a bit), key pairs are generated as follows: a key is randomly generated by retrieving the binary representation of a random integer between 0 and $2^{keylength}-1$; this first key is copied to a second key; a Bernoulli trial with probability equal to the error rate is executed for each bit of the second key². Note that even though this process is an acceptable simulation of the behaviour of a binary symmetric channel, it does not ensure that the keys have an error rate close to the one defined (while *improbable*, it is *possible* that the actual error rate in the key is far from the defined error rate, which is the channel error rate). In this implementation, the actual error rate is never used (although it is kept as a statistic); the channel error rate is used as the error rate for all purposes.

The datasets are saved in comma-separated values (CSV) files with header: *initial key, key with errors, channel error rate*; each key is kept in hexadecimal representation.

As an example, the command necessary to create a dataset of default size (10^5 key pairs) of keys with 16384 bits and simulating a BSC with 5% error rate (creating, by default, a file named *16384-005.csv*) is:

```
$ cascade-study create_dataset 16384 0.05
```

4.2.2 Algorithm Executor

The main functionality of this tool is the one that executes an algorithm and retrieves information about the run. This is done with the command:

```
$ cascade-study run_algorithm <algorithm> <dataset file> <options>*
```

The algorithm executor will run the algorithm for each line of the dataset (once, by default, but the number of lines to process and the number of runs by line can be altered with CLI utility options) and output a statistics file containing the following fields:

²This process is repeated until all the desired keys are generated.

- dataset file with the keys and error rate used for the run and corresponding line;
- channel error rate and actual error rate;
- correctness of the run (true if the keys have no errors in the end of the protocol, false otherwise);
- bit error rate;
- reconciliation efficiency;
- number of channel uses;
- total length of the exchanged message;
- the random seed used³;
- additional iteration data.

The possible algorithms are: *original*; *biconf*; *yanetal*; *option7*; *option8*. As an example, the command necessary to run the original algorithm for all keys in a dataset contained in the file *16384-005.csv* (creating, by default, a file named *original-16384-005.csv*) is:

```
$ cascade-study run_algorithm original 16384-005.csv
```

4.2.3 Algorithm Implementation

To implement multiple Cascade versions, a Object-Oriented approach was taken. The protocol has a common behaviour that is shared by all versions, the Binary protocol and the Cascade main loop (with the associated cascade effect). This behaviour was implemented in an abstract class.

The extending classes set the number of iterations and the strategy for the block generation. In the *biconf* version, it also includes some extra behaviour.

Although the processing of each algorithm was not parallelized as discussed earlier, the number of channel uses needs to be counted as though it was parallelized. The approach to this problem was: on each iteration, keep an array containing one array for each block of the iteration (and an integer containing the length of the initial parity exchange for that iteration); each inner array will contain one entry with the length of each message required to process that block (in case of a block with an error, this includes both parity exchanges from the binary protocol and from the consequent cascade effect). For each iteration, the length of largest array of each iteration is considered as the number of messages required in that iteration, since all other parity exchanges are independent of those, they could be sent with them. Fig. 4.1 shows the evolution of the array during the first iteration of an execution of Cascade. It is possible to see an initial parity exchange of 64 bits

³to allow verification reruns

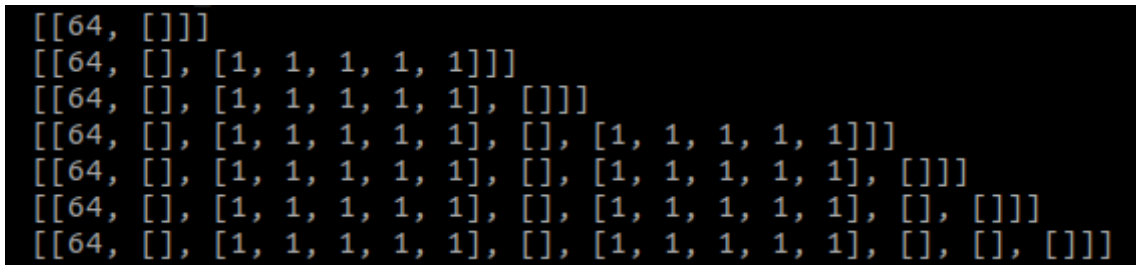


Figure 4.1: Evolution of the channel uses array during the first iteration.

(the key was split into 64 blocks) and that only the second and fourth blocks had odd numbers of errors and caused binary executions.

It is important to note that we consider that all parity exchanges in the processing of a block are dependent: including during the cascade effect, the priority queue elements are processed one at a time. Although this increases the number of channel uses it should allow for higher efficiency since it is always the smallest block possible being processed.

4.2.4 Run Validation

For the effect of allowing the validation of obtained results, the *replicate run* feature was implemented. The syntax follows:

```
$ cascade-study replicate_run <algorithm> <results file> <options>*
```

This command reruns the given algorithm with the keys and seeds defined in the given results file and outputs the run statistics, that can then be compared with the original results file, verifying the integrity of the results.

As an example, in order to validate the results in *original-16384-005.res.csv*, the following command should be executed:

```
$ cascade-study replicate_run original original-16384-005.res.csv
```

This will, by default, generate a file named *original-16384-005.res.replica.csv*. The sorting on this file is very likely not the same as the original file. Given this, one way of verifying the integrity of the results is the following bash command:

```
$ if [ "`sort original-16384-005.res.replica.csv`" == "`sort original
-16384-005.res.csv`" ]; then echo 'Valid results';
fi
```

4.2.5 Results Processing

The obtained results are divided among files for a given algorithm, key length and error rate. In order to analyze these results, we created the *process results* feature with the following syntax:

```
$ cascade-study process_results <files> <options>*
```

This will produce a file with one line for each processed file containing the average and variance for: the reconciliation efficiency, the bit error rate, the number of channel uses and the message length. It will also contain the frame error rate found in that run.

For the reconciliation efficiency, only the successful reconciliations are taken into account; for the BER only the unsuccessful ones.

To allow a better analysis of the processed results, a utility to present the data in charts was developed. It can be used with the following command:

```
$ cascade-study create_chart <input file> <x axis name> <y axis name>
[-vk <variance column name>]
[-l <line name> <column name> <line value>]*
[-r <column to restrict> <value to restrict>]*
<options>*
```

This command will use the data in the input file to create a chart, using the given column names to define the x and y axis. It is possible to set error bars by using the flag `-vk` with the name of the column containing the variance. The length of the error bars will be defined by two times the standard deviation, meaning that approximately 95%⁴ of the obtained values will be contained in that space.

It is possible to define multiple lines to be drawn by using the flag `-l` with the name to be attributed to the line and the name and value for the column that will be used to restrict the values used for the line. It is also possible to set global restrictions for all lines using the flag `-r` with the name of the column to restrict and value it should be restricted to. Among the remaining CLI options, there are flags to define the range for each axis and the distance between ticks.

To illustrate with examples, the command to process all results files contained in a folder *results*, outputting to a *all_results.csv* file, is:

```
$ cascade-study process_results results/*.res.csv -o all_results.csv
```

And the command to generate the chart for reconciliation efficiency as a function of the key length for keys with 5% error rate, plotting one line for each algorithm is:

⁴Assuming a normal distribution

```
$ cascade-study create_chart all_results.csv "key length" "avg eff" \  
-r "error rate" 0.05 \  
-l yanetal algorithm sugimoto \  
-l original algorithm original \  
-l biconf algorithm biconf \  
-l option7 algorithm option7 \  
-l option8 algorithm option8
```

4.2.6 Other Command Line options

All command line functionalities have utility options, like the output file name. The number of processor cores to use can also be defined, otherwise the program will use all available cores to parallelize the dataset creation, algorithm runs or their validation. The description of all command line options can be found in the documentation provided with the source code.

4.3 Block Parity Inference

With the objective of improving the reconciliation efficiency and reducing the number of channel uses without reducing the frame error rate, we propose an optimization to the Cascade protocol. It was implemented and the improvements resulting from it will be shown in the next chapter.

The optimization and its implementation will be described in the following sections.

4.3.1 Description

It was noticed that the Binary protocol could, eventually, request the parity of the same subblock twice in the same protocol execution. In fact, it could also request the parity of subblocks whose parity had not been requested but could be inferred. We propose a dynamic programming approach: all blocks (and subblocks) are kept, with the corresponding parities. If the parity of a block is already known or can be computed by a linear combination of any other known parities, it is not necessary to exchange it.

This poses a clear memory and computation trade-off as we have to: keep an ordered record of all exchanged blocks and their parities; before *each* parity exchange, perform a $O(n)$ search through the records to try to infer that parity.

4.3.2 Implementation

This optimization was implemented in C language and used in Python as a module by taking part of the interoperability between both languages. The choice of using C was made for scalability issues: an initial approach in Python was slow and required a large amount of memory. Although it is probably possible to make an efficient implementation in Python, the ease of using low-level memory management and bitwise operations in C weighted heavily on the decision.

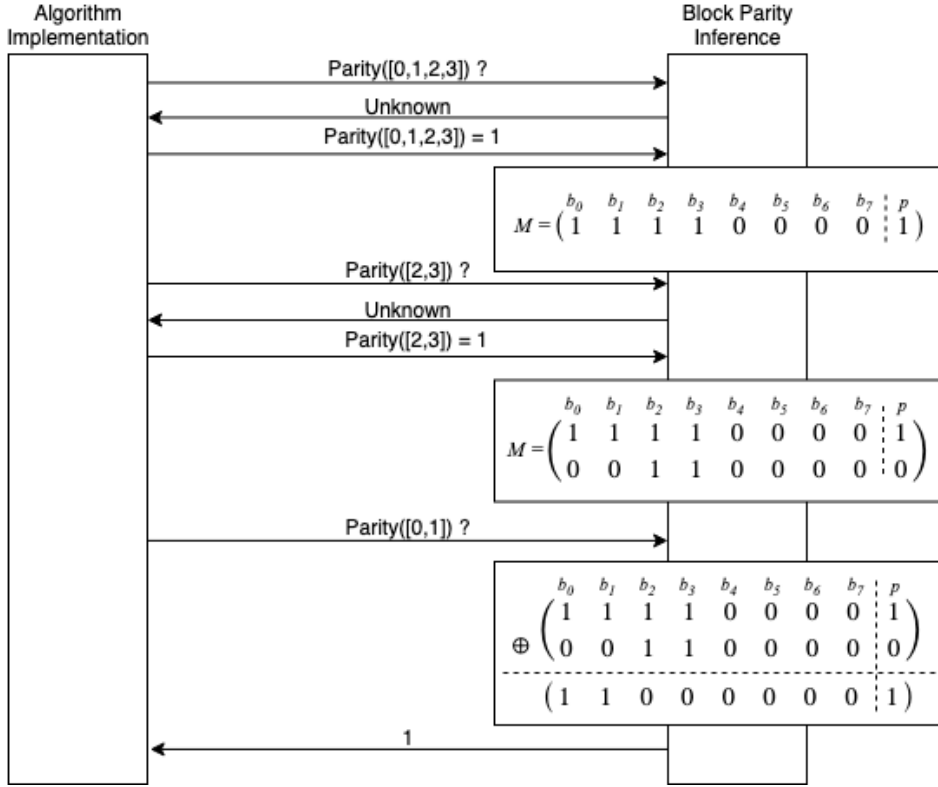


Figure 4.2: Evolution of the channel uses array during the first iteration.

The approach to the problem was to create an initially empty binary matrix with $n + 1$ rows, where n is the length of the key to reconcile. Each row in the matrix represents a block: a column i is 1 if the bit with index i is part of the block, 0 otherwise. The last column represents the parity of the block.

For each block that would be exchanged, we check if it is in the span of the matrix, that is, if its row representation (except the last column) can be generated by a linear combination (over modulo 2) of the existing rows in the matrix. If it can, then the last column of that linear combination is the parity of the block and there is no need to exchange a message. Otherwise, the parity is exchanged and the row representation with the corresponding parity is added to the matrix.

However, for this to be efficient, the insertions in the matrix have to be made while keeping it in row-echelon form (or lower triangular). This way, in order to check if a row ($target_row$) is in the span of the matrix: we create an initial row ($current_row$) with all zeros. For each row (row_i) in the matrix, being j the index of the first column of row_i with a bit set to 1, if the bit j of $target_row$ is different from the bit j of $current_row$, $current_row = current_row \oplus row_i$. The loop ends when there are no more rows in the matrix or when $current_row = target_row$.

A mock example of the usage of Block Parity Inference is shown in Fig. 4.2. In the figure, the parity check matrix, initially empty, is represented by M .

To integrate the Block Parity Inference optimization in the algorithms, a flag, $-bi$, can be set in the CLI when executing the *run algorithm* or the *replicate run* commands. Only if this flag is

set, the algorithm will run using this optimization.

4.4 Conclusion

This chapter presented a detailed problem description and comprehensive explanation of the features and implementation decisions of the program developed to study the Cascade protocol. It concludes with a description of the proposed optimization to the protocol, Block Parity Inference, and its implementation.

In the next chapter we will present the experiments performed with this tool and analyze the obtained results.

Chapter 5

Experiments and Results

This chapter will present the experiments ran with the developed tool. A thorough analysis of the obtained results follows.

5.1 Experiments

This study was performed on a wide range of datasets of different key lengths and error rates. For key lengths, we chose to use all powers of two (2^n) between 2^{10} (1024) bits and 2^{14} (16384) bits, inclusively, as it is standard in cryptography for key lengths and this range provides the more relevant (while computationally feasible to test) key lengths. We also chose to use 10000 bits as a key length to be able to directly compare with previous literature ([1, 3, 15, 16]). For range of error rates, like in [1], we used error rates up to 10%, since, in QKD, higher error rates usually mean that the protocol is aborted. For each key length, we generated datasets for key pairs with error rates from 0.5% until 9.5%, in steps of 0.5%. Given this, we will use the notation "full-step" to mean the integer percentage error rates (1%, 2%, ...).

Unfortunately, because of hardware limitations, it was not possible to execute the previously mentioned algorithms with datasets of 10^6 key pairs in the provided time for the full range of key lengths and error rates relevant to the study. Given this limitation, datasets of 10^5 key pairs were used. While the obtained results will be less precise than in [1] (especially for the FER and BER metrics), this will allow us to analyze a larger range of key lengths.

To analyze the presented Cascade versions and the effect of the Block Parity Inference (BPI) optimization, two experiments have been performed. The first experiment involved running all algorithms as previously described, for all generated datasets. The second experiment involved running all algorithms using the BPI optimization, for all datasets with full-step error rates¹. All analyzed cascade versions² with the corresponding adapted names and parameter description are

¹The cut on the other datasets was made because of the higher runtimes of the algorithms using the optimization

²Note that the option 8 protocol's block size k_3 was adapted to generate the same number of blocks for key lengths other than 16384

Experiments and Results

Table 5.1: Analyzed Cascade versions in both experiments

Protocol	Block Sizes (approx.)			Cascade passes	BICONF	Block Parity Inference	
	k_1	k_2	k_i			First Experiment	Second Experiment
original Ref.[3]	$0.73/Q$	$2k_1$	$2k_{i-1}$	4	no	no	yes
biconf Ref.[16]	$0.92/Q$	$3k_1$	-	2	yes	no	yes
Yan et al. Ref.[15]	$0.8/Q$	$5k_1$	$n/2$	10	no	no	yes
option7 Ref.[1]	$2^{\lceil \log_2 1/Q \rceil}$	$4k_1$	$n/2$	14	no	no	yes
option8 Ref.[1]	$2^{\lceil \alpha \rceil}$	$2^{\lceil (\alpha+12)/2 \rceil}$	$k_3 = n/4$ $k_i = n/2$	14	no	no	yes

$\alpha = \log_2(1/Q) - \frac{1}{2}$

displayed in table 5.1. Tables with the full compiled results for both experiments can be consulted in the Appendix A³

Over the next sections, we will present the results of these experiments and perform a individual analysis and a combined analysis to evidenciate the effect of the Block Parity Inference optimization. We will also compare with the results obtained in [1] using the subblock reuse optimization.

5.1.1 First experiment results

The results for the first experiment are in agreement with the ones presented in [1]. As can be seen in Fig 5.1⁴, the original version leaks significantly more information than the biconf or Yan et al's

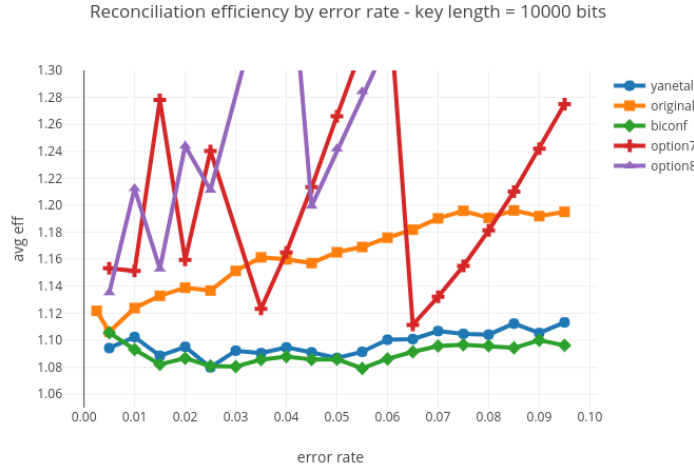


Figure 5.1: First experiment reconciliation efficiency by error rate on keys with 10000 bits

³The datasets, results and charts files are published in: thesis.andreireis.eu/data

⁴Charts for other key lengths follow the same pattern. For readability we will omit charts differing only on the key length or error rate if they represent the same results. They can be consulted on the mentioned link.

Experiments and Results

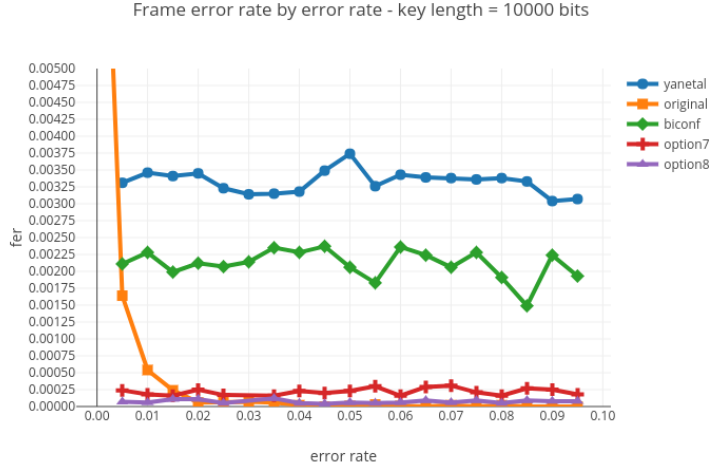


Figure 5.2: First experiment frame error rate by error rate on keys with 10000 bits

version and scales worse with the increasing error rate. Option7 and option8 are not remotely close to optimal efficiency - they were designed to take advantage of the subblock reuse optimization, without it, these versions are far from optimal both in terms of reconciliation efficiency and number of channel uses. Although the extra information exchange leads to small frame error rate (FER), the same order of FER can be achieved by the original algorithm with a smaller trade-off, as depicted in Fig 5.2.

Given this, we present Fig. 5.3 without the option7 and option8 versions to have a better window frame for the chart. It is possible to see that the number of channel uses metric shows different results from [1]. We attribute this to the point made earlier about different parallelization approaches. This statement is backed by the fact that the obtained results are coherent with the

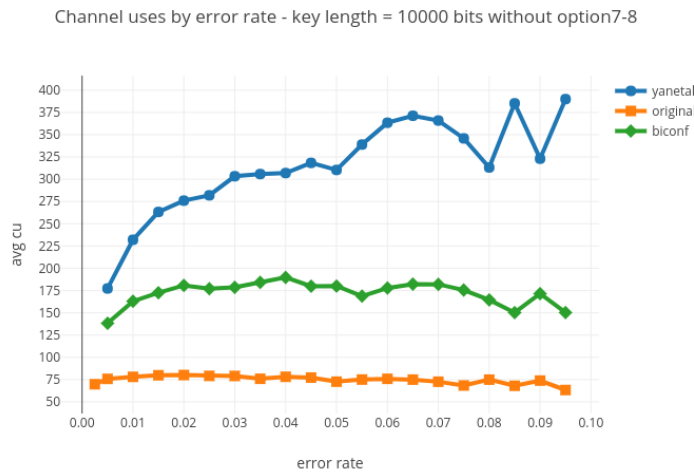


Figure 5.3: First experiment number of channel uses by error rate on keys with 10000 bits

Experiments and Results



Figure 5.4: Reconciliation efficiency by error rate for keys with 1024 and 2048 bits

ones showed in the paper: the comparison between the protocols is the same, only the order of magnitude of the results is higher.

This experiment shows that, between the analyzed versions (in table 5.1), the biconf protocol is the closest to optimal. Its efficiency improves with the length of the key, and although it is close to Yan et al's, it has lower frame error rate and number of channel uses throughout the whole range of keys and error rates. The point can be made that for small keys (< 4096 bits) the original version behaves better than the biconf version: it has lower FER and number of channel uses and for small error rates ($< 2.5\%$) the efficiency is also better, as can be seen in Figs 5.2 and 5.4. For higher error rates, the choice between the original and biconf versions is a trade-off of efficiency for probability of failure and number of channel uses.

This experiment also showed that the biconf and Yan et al's versions, with increasing key length, have a logarithmic decrease in reconciliation efficiency and logarithmic increase in the number of channel uses, as can be seen in Fig 5.5. These are indicators that the protocol is more efficient for larger key lengths.

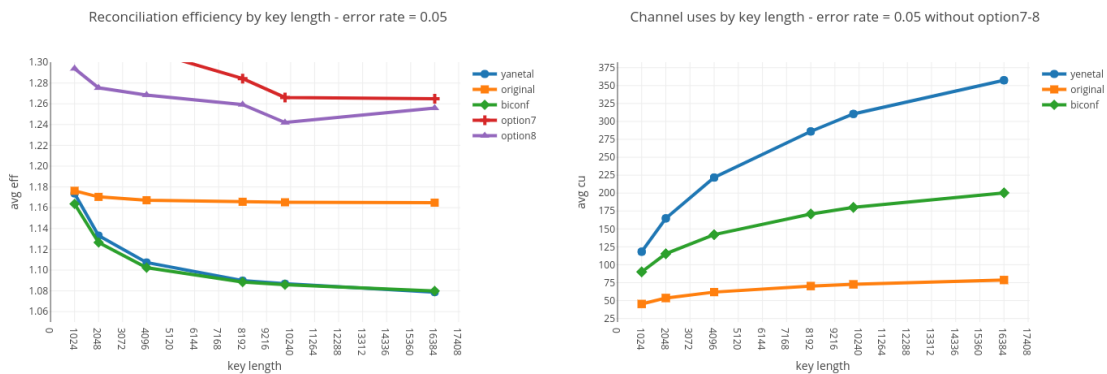


Figure 5.5: Reconciliation efficiency and channel uses by key length

Experiments and Results



Figure 5.6: Reconciliation efficiency by error rate on keys with 16384 bits

5.1.2 Second experiment results

The second experiment involved all algorithms using the Block Parity Inference optimization. In Fig 5.6 we can see that Yan et al's version is slightly better than the biconf version in terms of reconciliation efficiency. This is probably due to the biconf approach not taking advantage of the cascade effect after the second iteration which causes it to not take as much advantage of the optimization. A more detailed chart on the evolution of the reconciliation efficiency with the increase of the key length for each analyzed error rate is shown in Fig 5.7. This shows that for large key lengths and error rate of 6%, the biconf version behaves slightly better in terms of efficiency, as also evidenced in Fig 5.6.

In Figs 5.8, 5.9 and 5.10 we compare the results from both experiments using corresponding charts. As expected, the optimization does not impact the frame error rate, but it shows significant improvement in the reconciliation efficiency and a slight improvement in the number of channel uses for all algorithms.

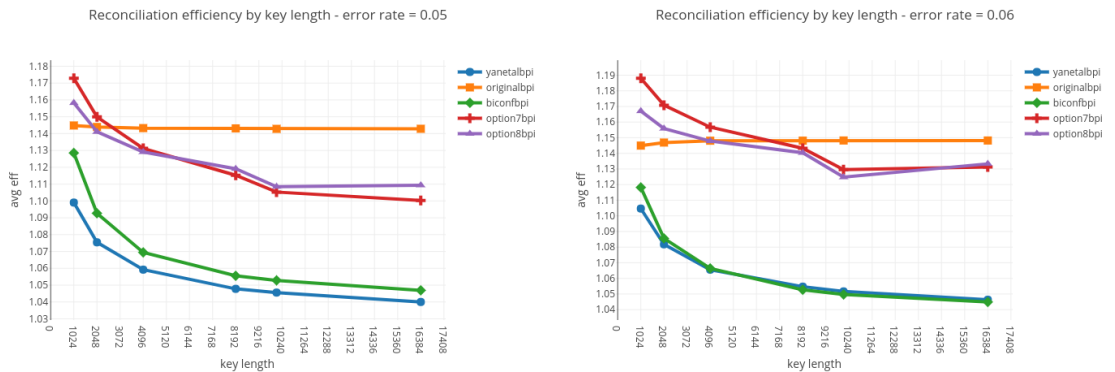


Figure 5.7: Reconciliation efficiency by key length with 5% error rate and 6% error rate

Experiments and Results



Figure 5.8: Reconciliation efficiency of the first experiment (on the left) and second (on the right)

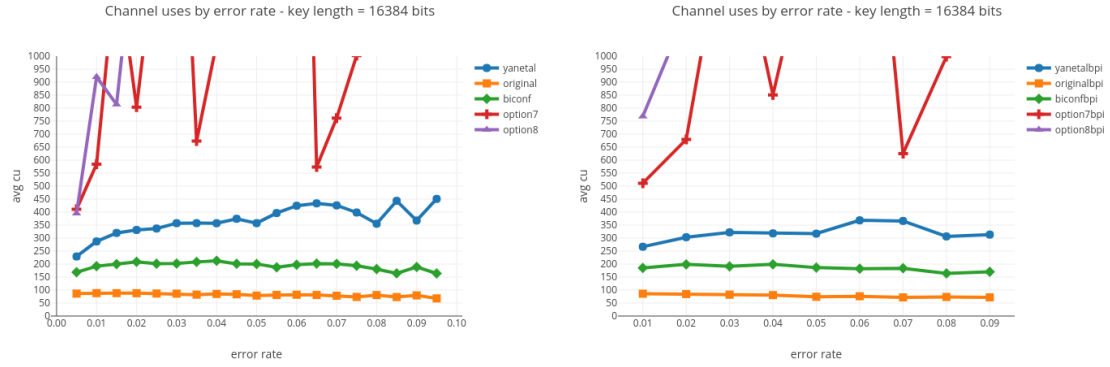


Figure 5.9: Channel uses of the first experiment (on the left) and second (on the right)

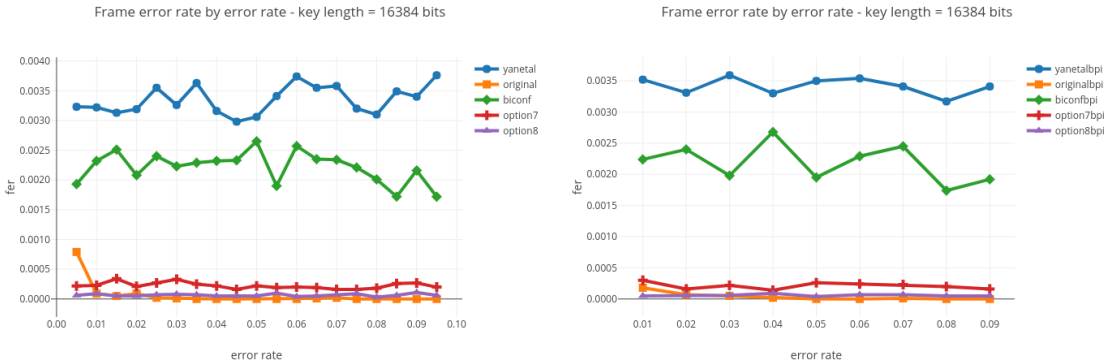


Figure 5.10: Frame error rate of the first experiment (on the left) and second (on the right)

It is worth to notice the significant improvement of both the option7 and option8 algorithms. We attribute this to the fact that these versions use powers of two for block sizes, allowing for a bigger capitalization on Block Parity Inference. While these block sizes also make the most of the dichotomic search in the binary protocol (the block with 2^n bits is the biggest block that can be corrected with n parity exchanges using binary), the distribution of errors between the blocks for

the initial iterations is not optimal. As such, the number of errors corrected in the initial iterations is comparatively low. A large number of errors is corrected in more advanced iterations and as these have large block sizes, a higher number of parity exchanges is required to correct them. In the original version of these protocols, using subblock reuse, this is avoided because in the more advanced iterations, subblocks from binary exchanges in the initial iterations are reused and the number of required parity exchanges to correct these errors is lower. This suggests that these versions could be very efficient using both optimizations (Block Parity Inference and subblock reuse).

Comparing with fig 5.11⁵, we can see that Yan et al’s version with our proposed optimization achieves a reconciliation efficiency very close to the version proposed as optimal in [1], option8 with the subblock reuse optimization (in fact, for some error rates it can have better efficiency). Unfortunately, due to the different parallelization approaches, comparing the number of channel uses of these versions holds no relevance.

5.2 Conclusions

In this chapter, we define the experiments ran and show the obtained results. We show that our proposed optimization effectively improves the reconciliation efficiency and number of channel uses for any of the studied algorithms without affecting their robustness.

With the results from both experiments we propose Yan et al’s protocol using the Block Parity Inference modification as the most optimized Cascade version. If the hardware is not sufficient for the memory and processing power trade-off, we propose the biconf protocol as the most optimized version⁶ from the ones studied.

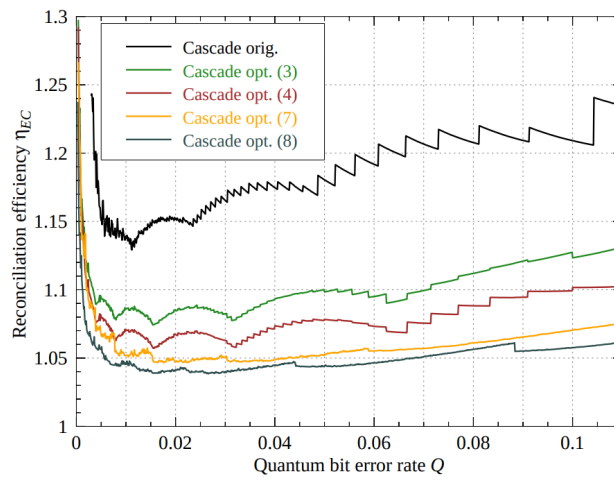


Figure 5.11: Reconciliation efficiency by error rate, from [1].

⁵orig., opt.(7) and opt.(8) correspond to original, option7 and option8. The results are for keys with length 10000 bits except for option8 where it is 16384 bits.

⁶Although, as previously mentioned, the original version may perform better in certain circumstances

Experiments and Results

We conclude that it would be very interesting to study an integration of the Block Parity Inference optimization with the subblock reuse optimization described in the literature. Theoretically, these two optimizations should complement each other: Block Parity Inference could allow to infer the smallest subblock to reuse and avoid exchanging unnecessary parities. This should improve the reconciliation efficiency and number of channel uses metrics of any cascade version even further without compromising its robustness, although with an even higher computation and memory penalty.

Chapter 6

Conclusions and Future Work

This chapter will present the conclusions of this dissertation, evidentiating its main contributions. Future work suggestions, based on what was observed in the experiments, follow.

6.1 Conclusions

This dissertation had the main objective of analyzing different Cascade versions, presented in current literature, in order to propose an optimal version.

To be able to do this, since there were no published practical implementations or examples, a tool was developed for that purpose. This software implements the Cascade versions discussed in this dissertation, allowing the execution of any experiments aiming to study the discussed metrics (reconciliation efficiency, frame error rate, bit error rate and number of channel uses). It also allows: the verification of results; simple processing and visualization of experiment data; simple extension, allowing to experiment different Cascade versions by tweaking protocol parameters or to modify the metrics to study. This publicly available software should reduce the overhead of future studies on the Cascade protocol.

During the process of analyzing the algorithms, we implemented an optimization, Block Parity Inference. We performed two experiments, one using the Block Parity Inference and one without it. This showed that our proposed optimization improved the reconciliation efficiency and reduced the number of channel uses of all studied Cascade versions, without altering the frame error rate, only with a memory and computation penalty.

The results of these experiments allow us to propose Yan et al's [15] Cascade using Block Parity Inference as the optimal version.

6.2 Future Work

The main track for future work that arose from this dissertation is the study of a combination of the Block Parity Inference and subblock reuse optimizations. An analysis over the same algorithms could allow the proposal of an even more optimized Cascade version.

Nonetheless, the lack of practical implementations or examples proved to be an issue during this work. As already mentioned, there are ambiguous implementation aspects that can have high impact on the studied metrics (e.g. the effect of parallelization in the number of channel uses). It should be interesting to extend the developed tool, implementing different approaches from which one could choose when running an algorithm, so that it would be possible to study the impact of these differences.

As previously mentioned, due to hardware limitations, this study was performed with 10^5 experiments for each algorithm, key length and error rate triplet. It should be possible to decrease the runtime by parallelizing computation using threads (current parallelization is limited to executing multiple algorithm runs in multiple processes), thus allowing a study with a higher precision by running at least 10^6 experiments for each triplet. This would also allow to run the study for larger key rates ($> 2^{14}$ bits).

References

- [1] Jesus Martinez-Mateo, Christoph Pacher, Momtchil Peev, Alex Ciurana, and Vicente Martin. Demystifying the Information Reconciliation Protocol Cascade. *arXiv:1407.3257 [quant-ph]*, Jul 2014.
- [2] Charles H. Bennett and Gilles Brassard. Quantum Cryptography: Public Key Distribution and Coin Tossing. *Proceedings of IEEE International Conference on Computers, Systems and Signal Processing*, pages 175–179, Sep 1984.
- [3] Gilles Brassard and Louis Salvail. Secret-Key Reconciliation by Public Discussion. In *Advances in Cryptology — EUROCRYPT '93*, pages 410–423. Springer Berlin Heidelberg, Berlin, Heidelberg, 1993.
- [4] C. E. Shannon. Communication Theory of Secrecy Systems. *Bell System Technical Journal*, 28(4):656 – 715, Oct 1949.
- [5] Frank Miller. *Telegraphic Code to Insure Privacy and Secrecy in the Transmission of Telegrams*. C.M. Cornwell, 1882.
- [6] C. E. Shannon. A Mathematical Theory of Communication. *Bell System Technical Journal*, 1948.
- [7] David Slepian, Jack K. Wolf, and David Slepian. Noiseless Coding of Correlated Information Sources. *IEEE Transactions on Information Theory*, 19(4):471 – 480, July 1973.
- [8] Wolfgang Tittel, Hugo Zbinden, and Nicolas Gisin. Quantum cryptography. 74(January):145–195, 2002.
- [9] Valerio Scarani, Nicolas J Cerf, and Norbert Lütkenhaus. The security of practical quantum key distribution. *Rev. Mod. Phys.*, 81(September):1301–1350, 2009.
- [10] Artur K. Ekert. Quantum cryptography based on Bell’s theorem. *Physical Review Letters*, 67, 1991.
- [11] Whitfield Diffie and Martin E. Hellman. New Directions in Cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976.

REFERENCES

- [12] Chi-hang Fred Fung, Xiongfeng Ma, and H F Chau. Practical issues in quantum-key-distribution postprocessing. *Phys. Rev. A*, 81, 2010.
- [13] D. R. Stinson. Universal Hash Families and the Leftover Hash Lemma, and Applications to Cryptography and Computing. *Journal of Combinatorial Mathematics and Combinatorial Computing*, 42:3 – 31, 2002.
- [14] Charles H. Bennett, François Bessette, Gilles Brassard, Louis Salvail, and John Smolin. Experimental quantum cryptography. *Journal of Cryptology*, 5, 1992.
- [15] Hao Yan, Tienan Ren, Xiang Peng, Xiaxiang Lin, Wei Jiang, Tian Liu, and Hong Guo. Information reconciliation protocol in quantum key distribution system. In *Proceedings - 4th International Conference on Natural Computation, ICNC 2008*, 2008.
- [16] Tomohiro Sugimoto and Kouichi Yamazaki. A study on secret key reconciliation protocol "cascade". *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, E83-A(10), 2000.
- [17] Shengli Liu, Henk C.A. Van Tilborg, and Marten Van Dijk. A practical protocol for advantage distillation and information reconciliation. *Designs, Codes, and Cryptography*, 30:39 – 62, Aug 2003.

Appendix A

Appendix 1

Table A.1: First Experiment Results

biconf	1024	0.005	1.3210	0.2065	0.0010	0.0031	2.45e-06	36.9025	193.646	61.4217	446.606
biconf	1024	0.015	1.2099	0.0777	0.0012	0.0042	7.06e-06	63.7934	502.336	139.192	1029.42
biconf	1024	0.01	1.2487	0.1124	0.0010	0.0038	5.56e-06	53.8090	383.661	103.291	769.033
biconf	1024	0.025	1.1837	0.0486	0.0016	0.0041	8.46e-06	75.5552	661.091	204.412	1449.20
biconf	1024	0.02	1.2016	0.0602	0.0014	0.0046	7.86e-06	71.6003	600.987	174.024	1262.37
biconf	1024	0.035	1.1792	0.0367	0.0016	0.0054	1.40e-05	83.9163	818.782	264.260	1844.19
biconf	1024	0.03	1.1771	0.0408	0.0017	0.0050	1.28e-05	78.9999	726.386	234.286	1618.46
biconf	1024	0.045	1.1703	0.0288	0.0019	0.0057	2.00e-05	87.6780	900.538	317.260	2120.24
biconf	1024	0.04	1.1753	0.0333	0.0016	0.0054	1.30e-05	87.9863	894.338	291.596	2051.32
biconf	1024	0.055	1.1529	0.0226	0.0016	0.0058	2.29e-05	86.6674	910.280	362.738	2233.92
biconf	1024	0.05	1.1637	0.0263	0.0016	0.0060	2.00e-05	90.0447	951.625	341.247	2257.94
biconf	1024	0.065	1.1618	0.0202	0.0014	0.0057	1.77e-05	92.8159	1045.35	412.789	2550.77
biconf	1024	0.06	1.1572	0.0214	0.0019	0.0064	2.43e-05	90.7247	994.289	387.981	2408.95
biconf	1024	0.075	1.1611	0.0172	0.0017	0.0060	1.78e-05	92.7626	1060.22	456.932	2667.42
biconf	1024	0.07	1.1635	0.0189	0.0018	0.0066	3.04e-05	94.1261	1081.01	435.941	2649.75
biconf	1024	0.085	1.1514	0.0133	0.0015	0.0060	2.12e-05	85.7047	896.686	494.649	2458.28
biconf	1024	0.08	1.1607	0.0158	0.0017	0.0071	2.65e-05	90.9690	1034.02	477.991	2683.99
biconf	1024	0.095	1.1496	0.0122	0.0014	0.0065	2.72e-05	87.2267	945.273	533.202	2627.20
biconf	1024	0.09	1.1613	0.0145	0.0020	0.0070	3.03e-05	94.4069	1125.56	518.998	2894.55
biconf	2048	0.005	1.2213	0.1110	0.0010	0.0017	1.00e-06	59.0245	484.022	113.572	960.503
biconf	2048	0.01	1.1737	0.0576	0.0014	0.0021	1.68e-06	79.8312	759.731	194.185	1576.80
biconf	2048	0.025	1.1373	0.0248	0.0016	0.0032	6.42e-06	104.287	1250.44	392.831	2962.52
biconf	2048	0.02	1.1494	0.0320	0.0018	0.0028	3.94e-06	101.099	1180.57	332.904	2685.30

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
biconf	2048	0.035	1.1341	0.0178	0.0019	0.0030	6.24e-06	111.105	1467.16	508.334	3575.94
biconf	2048	0.03	1.1314	0.0206	0.0017	0.0031	7.60e-06	106.417	1348.09	450.390	3266.63
biconf	2048	0.045	1.1310	0.0137	0.0019	0.0032	5.97e-06	113.660	1562.17	613.236	4023.39
biconf	2048	0.04	1.1369	0.0159	0.0021	0.0034	6.99e-06	115.781	1611.79	564.080	3919.60
biconf	2048	0.055	1.1167	0.0104	0.0016	0.0035	7.67e-06	110.264	1499.28	702.721	4132.94
biconf	2048	0.05	1.1266	0.0122	0.0020	0.0036	7.49e-06	115.360	1610.12	660.752	4203.61
biconf	2048	0.065	1.1282	0.0094	0.0018	0.0034	6.99e-06	118.508	1760.94	801.684	4761.86
biconf	2048	0.06	1.1228	0.0098	0.0020	0.0035	7.28e-06	114.954	1635.61	752.949	4429.55
biconf	2048	0.075	1.1285	0.0079	0.0020	0.0033	6.24e-06	117.173	1735.95	888.181	4890.32
biconf	2048	0.07	1.1309	0.0087	0.0022	0.0034	7.11e-06	118.882	1799.28	847.444	4910.50
biconf	2048	0.085	1.1215	0.0060	0.0016	0.0033	7.41e-06	104.457	1333.07	963.611	4403.84
biconf	2048	0.08	1.1278	0.0070	0.0017	0.0032	5.74e-06	111.996	1570.99	928.963	4769.10
biconf	2048	0.095	1.1217	0.0053	0.0016	0.0037	1.19e-05	105.531	1369.70	1040.50	4561.38
biconf	2048	0.09	1.1311	0.0065	0.0020	0.0035	1.05e-05	116.633	1737.10	1011.09	5153.59
biconf	4096	0.005	1.1492	0.0572	0.0012	0.0010	4.59e-07	88.3894	961.279	213.743	1979.73
biconf	4096	0.015	1.1124	0.0196	0.0018	0.0016	1.41e-06	123.904	1758.53	511.898	4155.06
biconf	4096	0.01	1.1299	0.0307	0.0019	0.0014	8.82e-07	113.686	1495.71	373.867	3359.20
biconf	4096	0.025	1.1044	0.0118	0.0019	0.0017	1.96e-06	134.613	2140.73	762.911	5657.44
biconf	4096	0.02	1.1152	0.0152	0.0021	0.0015	1.75e-06	133.574	2068.49	646.038	5092.95
biconf	4096	0.035	1.1057	0.0083	0.0020	0.0017	2.48e-06	141.522	2407.23	991.245	6693.51
biconf	4096	0.03	1.1019	0.0095	0.0021	0.0018	2.37e-06	135.961	2206.24	877.312	6039.07
biconf	4096	0.045	1.1037	0.0061	0.0021	0.0016	2.17e-06	141.117	2407.98	1196.93	7210.68

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
biconf	4096	0.04	1.1088	0.0073	0.0019	0.0019	2.67e-06	145.861	2582.82	1100.35	7238.17
biconf	4096	0.055	1.0944	0.0046	0.0019	0.0019	2.32e-06	134.910	2203.34	1377.41	7351.38
biconf	4096	0.05	1.1023	0.0055	0.0022	0.0019	2.81e-06	141.992	2447.56	1293.07	7520.29
biconf	4096	0.065	1.1063	0.0042	0.0021	0.0018	3.66e-06	145.195	2653.94	1572.27	8466.99
biconf	4096	0.06	1.1032	0.0045	0.0021	0.0018	2.20e-06	142.333	2517.42	1479.58	8045.13
biconf	4096	0.075	1.1103	0.0035	0.0021	0.0020	2.69e-06	142.131	2520.20	1747.74	8650.62
biconf	4096	0.07	1.1111	0.0039	0.0021	0.0018	2.22e-06	145.980	2708.49	1665.33	8755.42
biconf	4096	0.085	1.1056	0.0026	0.0016	0.0018	2.90e-06	123.979	1803.63	1899.92	7684.36
biconf	4096	0.08	1.1090	0.0031	0.0020	0.0020	2.68e-06	134.353	2213.42	1826.79	8335.13
biconf	4096	0.095	1.1067	0.0023	0.0018	0.0016	2.12e-06	124.444	1862.01	2053.28	8000.30
biconf	4096	0.09	1.1129	0.0028	0.0022	0.0019	2.27e-06	139.969	2453.69	1989.59	9011.47
biconf	8192	0.005	1.1132	0.0294	0.0019	0.0007	2.44e-07	126.478	1835.58	414.090	4077.12
biconf	8192	0.015	1.0874	0.0095	0.0020	0.0008	5.33e-07	160.875	3018.34	1000.89	8047.90
biconf	8192	0.01	1.1000	0.0147	0.0020	0.0009	5.06e-07	151.102	2637.55	727.977	6424.05
biconf	8192	0.025	1.0852	0.0054	0.0021	0.0009	5.58e-07	167.215	3271.24	1499.31	10229.63
biconf	8192	0.02	1.0914	0.0070	0.0023	0.0010	7.80e-07	169.205	3325.01	1264.53	9429.73
biconf	8192	0.035	1.0899	0.0038	0.0021	0.0010	8.08e-07	174.566	3607.18	1954.14	12183.11
biconf	8192	0.03	1.0845	0.0044	0.0021	0.0009	6.43e-07	168.773	3352.43	1727.01	11105.25
biconf	8192	0.045	1.0885	0.0027	0.0020	0.0010	9.78e-07	170.576	3388.06	2360.88	12937.32
biconf	8192	0.04	1.0920	0.0033	0.0024	0.0010	1.07e-06	179.310	3855.26	2167.53	13057.77
biconf	8192	0.055	1.0815	0.0021	0.0019	0.0008	4.81e-07	160.993	2995.80	2722.39	13130.10
biconf	8192	0.05	1.0885	0.0024	0.0020	0.0010	7.47e-07	170.802	3405.21	2553.80	13445.47

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
biconf	8192	0.065	1.0941	0.0019	0.0024	0.0011	8.04e-07	173.412	3643.79	3110.04	14985.67
biconf	8192	0.06	1.0890	0.0020	0.0019	0.0010	6.76e-07	169.026	3386.38	2921.23	14175.64
biconf	8192	0.075	1.0988	0.0015	0.0022	0.0011	8.64e-07	167.220	3292.93	3459.26	15045.30
biconf	8192	0.07	1.0981	0.0017	0.0022	0.0012	1.47e-06	172.901	3616.42	3291.60	15413.30
biconf	8192	0.085	1.0966	0.0012	0.0018	0.0008	5.20e-07	144.350	2302.01	3768.90	13763.21
biconf	8192	0.08	1.0978	0.0014	0.0019	0.0010	9.53e-07	157.222	2889.23	3616.83	14667.06
biconf	8192	0.095	1.0978	0.0010	0.0018	0.0010	5.57e-07	143.694	2281.95	4073.30	14098.58
biconf	8192	0.09	1.1025	0.0012	0.0020	0.0010	1.13e-06	164.285	3246.38	3942.13	15715.98
biconf	10000	0.005	1.1056	0.0240	0.0021	0.0006	2.44e-07	138.205	2209.23	502.041	4958.61
biconf	10000	0.015	1.0821	0.0076	0.0020	0.0008	5.51e-07	172.459	3480.17	1215.82	9537.52
biconf	10000	0.01	1.0932	0.0117	0.0023	0.0008	3.75e-07	162.929	3035.88	883.150	7656.25
biconf	10000	0.025	1.0809	0.0042	0.0021	0.0009	5.72e-07	177.107	3649.14	1823.07	12074.49
biconf	10000	0.02	1.0866	0.0057	0.0021	0.0008	4.01e-07	180.735	3859.41	1536.83	11330.01
biconf	10000	0.035	1.0856	0.0030	0.0023	0.0008	5.87e-07	184.213	3996.14	2376.15	14301.51
biconf	10000	0.03	1.0805	0.0035	0.0021	0.0008	4.39e-07	178.597	3758.76	2100.31	13122.30
biconf	10000	0.045	1.0857	0.0022	0.0024	0.0009	4.95e-07	179.927	3821.77	2874.47	15446.58
biconf	10000	0.04	1.0879	0.0026	0.0023	0.0008	4.58e-07	189.703	4316.13	2635.78	15427.06
biconf	10000	0.055	1.0791	0.0017	0.0018	0.0008	4.34e-07	168.712	3317.06	3315.87	15595.21
biconf	10000	0.05	1.0859	0.0020	0.0021	0.0009	5.91e-07	180.077	3798.74	3110.07	16002.51
biconf	10000	0.065	1.0914	0.0015	0.0022	0.0009	6.69e-07	182.189	3970.73	3786.89	17647.23
biconf	10000	0.06	1.0862	0.0016	0.0024	0.0008	6.06e-07	177.806	3726.99	3556.67	16820.66
biconf	10000	0.075	1.0966	0.0012	0.0023	0.0008	6.43e-07	175.538	3684.99	4214.29	18020.37

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
biconf	1000	0.07	1.0957	0.0014	0.0021	0.0009	6.76e-07	181.973	4035.38	4009.43	18196.07
biconf	1000	0.085	1.0943	0.0009	0.0015	0.0008	5.21e-07	150.233	2461.13	4591.17	16373.25
biconf	1000	0.08	1.0958	0.0011	0.0019	0.0008	5.48e-07	164.471	3086.83	4407.21	17275.61
biconf	1000	0.095	1.0962	0.0008	0.0019	0.0007	3.55e-07	150.282	2504.59	4965.22	16830.94
biconf	1000	0.09	1.1000	0.0010	0.0022	0.0008	4.66e-07	171.498	3463.51	4801.20	18482.17
biconf	16384	0.005	1.0882	0.0142	0.0019	0.0004	1.06e-07	168.609	3196.37	809.646	7845.80
biconf	16384	0.015	1.0721	0.0043	0.0025	0.0005	1.85e-07	199.913	4539.33	1973.51	14730.96
biconf	16384	0.01	1.0795	0.0068	0.0023	0.0005	1.77e-07	191.880	4179.70	1428.95	12005.57
biconf	16384	0.025	1.0732	0.0024	0.0024	0.0005	2.58e-07	201.350	4502.35	2965.67	18350.73
biconf	16384	0.02	1.0771	0.0032	0.0021	0.0005	2.37e-07	208.540	4957.74	2496.00	17258.21
biconf	16384	0.035	1.0790	0.0017	0.0023	0.0006	2.53e-07	208.192	4888.61	3869.42	21926.57
biconf	16384	0.03	1.0725	0.0020	0.0022	0.0005	1.75e-07	202.262	4593.52	3415.68	19967.85
biconf	16384	0.045	1.0803	0.0012	0.0023	0.0006	2.86e-07	201.138	4399.04	4686.03	23177.32
biconf	16384	0.04	1.0812	0.0015	0.0023	0.0006	3.16e-07	212.691	5140.87	4292.17	23386.57
biconf	16384	0.055	1.0743	0.0009	0.0019	0.0005	2.12e-07	187.452	3769.38	5408.37	23627.89
biconf	16384	0.05	1.0799	0.0011	0.0027	0.0005	2.62e-07	200.337	4373.35	5067.42	24059.21
biconf	16384	0.065	1.0866	0.0008	0.0023	0.0005	2.61e-07	201.832	4495.83	6177.07	26560.85
biconf	16384	0.06	1.0811	0.0009	0.0026	0.0005	2.28e-07	197.428	4297.75	5799.97	25555.18
biconf	16384	0.075	1.0925	0.0007	0.0022	0.0005	1.64e-07	193.605	4097.33	6878.74	27250.27
biconf	16384	0.07	1.0908	0.0008	0.0023	0.0005	2.30e-07	200.798	4421.64	6539.49	27197.93
biconf	16384	0.085	1.0911	0.0005	0.0017	0.0005	1.85e-07	164.419	2731.70	7500.24	24879.95
biconf	16384	0.08	1.0915	0.0006	0.0020	0.0005	2.33e-07	180.708	3478.92	7192.55	26464.68

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
biconf	16384	0.095	1.0927	0.0005	0.0017	0.0005	1.64e-07	163.988	2726.26	8108.78	25685.47
biconf	16384	0.09	1.0962	0.0005	0.0022	0.0005	2.08e-07	188.726	3929.47	7839.10	28094.92
option7	1024	0.005	1.5778	0.1869	0.0003	0.0037	3.99e-06	50.9516	417.016	73.3713	404.305
option7	1024	0.015	1.3947	0.0864	0.0004	0.0079	1.70e-05	119.963	1245.87	160.435	1146.18
option7	1024	0.01	1.4048	0.1087	0.0004	0.0045	7.27e-06	77.2609	789.614	116.212	744.552
option7	1024	0.025	1.3543	0.0602	0.0004	0.0105	3.96e-05	161.592	2001.78	233.857	1797.92
option7	1024	0.02	1.3269	0.0653	0.0003	0.0095	1.88e-05	121.709	1501.35	192.167	1370.25
option7	1024	0.035	1.2671	0.0411	0.0003	0.0103	4.07e-05	146.901	2503.05	283.967	2067.19
option7	1024	0.03	1.3883	0.0554	0.0002	0.0157	4.56e-05	203.142	2444.89	276.311	2201.24
option7	1024	0.045	1.3286	0.0412	0.0003	0.0180	0.0001	217.293	3860.34	360.170	3035.89
option7	1024	0.04	1.2962	0.0411	0.0003	0.0130	9.91e-05	180.931	3182.94	321.560	2534.35
option7	1024	0.055	1.4009	0.0405	0.0002	0.0234	0.0001	294.869	5092.26	440.722	4018.40
option7	1024	0.05	1.3644	0.0409	0.0002	0.0149	0.0001	255.387	4484.87	400.102	3519.01
option7	1024	0.065	1.2277	0.0247	0.0001	0.0085	2.27e-05	169.415	4197.59	436.197	3122.88
option7	1024	0.06	1.4343	0.0396	0.0003	0.0265	0.0002	334.309	5620.69	480.848	4464.19
option7	1024	0.075	1.2744	0.0280	0.0003	0.0147	0.0001	224.265	6281.74	501.497	4333.36
option7	1024	0.07	1.2498	0.0262	0.0002	0.0125	6.40e-05	195.740	5182.62	468.288	3682.39
option7	1024	0.085	1.3276	0.0305	0.0003	0.0234	0.0001	286.130	8449.71	570.340	5642.82
option7	1024	0.08	1.3005	0.0292	0.0003	0.0173	0.0002	254.298	7318.20	535.550	4955.43
option7	1024	0.095	1.3861	0.0329	0.0003	0.0271	0.0002	354.010	10574.91	642.825	7086.19
option7	1024	0.09	1.3566	0.0320	0.0002	0.0218	0.0002	319.447	9603.66	606.303	6392.56
option7	2048	0.005	1.3600	0.1085	0.0003	0.0025	2.49e-06	84.8549	993.671	126.478	939.011

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
option7	2048	0.015	1.3353	0.0522	0.0002	0.0070	1.51e-05	226.985	3070.15	307.233	2767.38
option7	2048	0.01	1.2829	0.0626	0.0003	0.0033	5.11e-06	135.195	1869.83	212.251	1713.59
option7	2048	0.025	1.3065	0.0371	0.0002	0.0068	2.51e-05	288.840	5637.38	451.262	4427.13
option7	2048	0.02	1.2525	0.0388	0.0003	0.0065	1.36e-05	205.232	4067.18	362.781	3262.92
option7	2048	0.035	1.2053	0.0235	0.0003	0.0065	1.84e-05	226.644	6620.53	540.253	4725.56
option7	2048	0.03	1.3630	0.0356	0.0003	0.0140	5.06e-05	377.553	7148.89	542.536	5669.34
option7	2048	0.045	1.2908	0.0269	0.0003	0.0128	7.09e-05	368.588	11983.80	699.852	7912.01
option7	2048	0.04	1.2466	0.0252	0.0002	0.0106	4.83e-05	294.146	9303.22	618.534	6212.42
option7	2048	0.055	1.3792	0.0284	0.0003	0.0163	0.0001	528.454	16976.07	867.828	11277.84
option7	2048	0.05	1.3347	0.0278	0.0002	0.0168	8.54e-05	446.608	14557.35	782.795	9595.17
option7	2048	0.065	1.1784	0.0137	0.0002	0.0084	4.19e-05	244.038	10194.51	837.351	6906.24
option7	2048	0.06	1.4235	0.0285	0.0002	0.0278	0.0001	612.219	19139.84	954.549	12875.54
option7	2048	0.075	1.2281	0.0170	0.0001	0.0111	5.92e-05	343.577	17511.15	966.615	10553.68
option7	2048	0.07	1.2024	0.0154	0.0002	0.0079	5.38e-05	291.082	13690.79	901.074	8626.02
option7	2048	0.085	1.2882	0.0209	0.0003	0.0186	9.92e-05	463.355	26438.48	1106.77	15457.57
option7	2048	0.08	1.2578	0.0191	0.0002	0.0150	0.0001	401.914	21989.23	1035.93	12963.74
option7	2048	0.095	1.3548	0.0244	0.0003	0.0260	0.0001	599.622	35854.03	1256.60	21042.54
option7	2048	0.09	1.3215	0.0227	0.0002	0.0146	9.73e-05	530.280	31141.39	1181.23	18152.12
option7	4096	0.005	1.2512	0.0619	0.0002	0.0019	9.98e-07	149.257	2326.52	232.723	2141.87
option7	4096	0.015	1.3120	0.0320	0.0003	0.0058	1.35e-05	420.453	8598.53	603.757	6798.99
option7	4096	0.01	1.2202	0.0368	0.0003	0.0026	2.54e-06	229.075	5005.54	403.756	4034.82
option7	4096	0.025	1.2809	0.0242	0.0002	0.0077	2.35e-05	503.326	17861.40	884.829	11580.53

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
option7	4096	0.02	1.2088	0.0229	0.0003	0.0052	1.06e-05	333.011	11500.28	700.252	7686.50
option7	4096	0.035	1.1663	0.0134	0.0002	0.0042	1.06e-05	338.182	17256.63	1045.55	10813.96
option7	4096	0.03	1.3534	0.0242	0.0003	0.0114	2.67e-05	688.502	23328.22	1077.47	15393.56
option7	4096	0.045	1.2600	0.0181	0.0002	0.0102	3.26e-05	608.048	38136.39	1366.29	21357.83
option7	4096	0.04	1.2102	0.0157	0.0002	0.0065	2.19e-05	463.636	27050.35	1200.98	15496.21
option7	4096	0.055	1.3652	0.0217	0.0003	0.0137	7.35e-05	936.004	59943.75	1718.10	34517.11
option7	4096	0.05	1.3113	0.0201	0.0002	0.0132	4.67e-05	765.836	49251.28	1538.16	27724.65
option7	4096	0.065	1.1447	0.0071	0.0001	0.0024	6.44e-06	338.399	22516.07	1626.88	14395.55
option7	4096	0.06	1.4161	0.0226	0.0002	0.0214	7.10e-05	1109.68	69060.04	1899.18	40786.14
option7	4096	0.075	1.1946	0.0103	0.0002	0.0058	3.25e-05	511.198	46302.09	1880.35	25462.34
option7	4096	0.07	1.1677	0.0085	0.0003	0.0062	2.67e-05	418.184	32916.58	1750.12	19139.46
option7	4096	0.085	1.2566	0.0143	0.0002	0.0085	5.95e-05	734.156	79805.19	2159.48	42376.41
option7	4096	0.08	1.2235	0.0122	0.0002	0.0110	6.09e-05	614.988	61653.72	2015.46	33151.82
option7	4096	0.095	1.3265	0.0186	0.0003	0.0175	0.0001	996.448	118809.89	2460.74	64106.93
option7	4096	0.09	1.2908	0.0166	0.0002	0.0133	5.73e-05	860.985	99012.22	2307.49	52970.48
option7	8192	0.005	1.1958	0.0349	0.0003	0.0014	8.56e-07	252.957	6009.89	444.838	4830.02
option7	8192	0.015	1.3039	0.0211	0.0002	0.0061	7.34e-06	764.556	27492.64	1200.08	17960.31
option7	8192	0.01	1.1814	0.0212	0.0002	0.0021	3.48e-06	371.082	13922.14	781.900	9287.46
option7	8192	0.025	1.2619	0.0166	0.0002	0.0066	1.56e-05	860.970	58556.41	1743.37	31705.99
option7	8192	0.02	1.1782	0.0138	0.0003	0.0033	5.66e-06	524.902	33202.63	1365.05	18614.91
option7	8192	0.035	1.1373	0.0073	0.0002	0.0020	4.63e-06	485.359	41203.47	2039.21	23370.83
option7	8192	0.03	1.3468	0.0182	0.0002	0.0100	2.68e-05	1242.18	81979.73	2144.54	46312.64

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
option7	8192	0.045	1.2341	0.0127	0.0002	0.0083	3.57e-05	984.200	119730.26	2676.67	59875.31
option7	8192	0.04	1.1823	0.0098	0.0002	0.0047	1.64e-05	711.038	75569.34	2346.64	38464.01
option7	8192	0.055	1.3486	0.0176	0.0002	0.0150	4.46e-05	1631.77	212663.54	3394.32	111945.08
option7	8192	0.05	1.2843	0.0152	0.0002	0.0072	4.45e-05	1291.91	164285.33	3012.98	83604.94
option7	8192	0.065	1.1210	0.0035	0.0003	0.0030	6.35e-06	448.779	44285.60	3186.27	28423.32
option7	8192	0.06	1.4013	0.0196	0.0003	0.0180	7.66e-05	1987.69	258389.69	3758.36	141406.24
option7	8192	0.075	1.1669	0.0058	0.0002	0.0075	3.90e-05	730.112	110460.12	3673.62	57391.99
option7	8192	0.07	1.1377	0.0044	0.0002	0.0018	3.33e-06	575.413	71484.78	3410.28	39968.97
option7	8192	0.085	1.2268	0.0094	0.0003	0.0066	3.26e-05	1121.28	222733.17	4216.33	111352.62
option7	8192	0.08	1.1914	0.0075	0.0003	0.0085	3.79e-05	911.822	161822.99	3924.95	81836.36
option7	8192	0.095	1.2971	0.0141	0.0003	0.0113	7.37e-05	1609.42	377181.08	4812.67	194610.21
option7	8192	0.09	1.2573	0.0116	0.0002	0.0110	5.71e-05	1355.33	294071.12	4495.32	148913.55
option7	10000	0.005	1.1533	0.0296	0.0002	0.0013	7.60e-07	286.332	8009.44	523.732	6100.72
option7	10000	0.015	1.2781	0.0193	0.0002	0.0063	5.30e-06	876.831	41094.51	1435.99	24395.32
option7	10000	0.01	1.1514	0.0174	0.0002	0.0014	1.52e-06	410.175	18287.56	930.184	11353.20
option7	10000	0.025	1.2403	0.0150	0.0002	0.0052	1.03e-05	972.129	84618.28	2091.75	42811.65
option7	10000	0.02	1.1596	0.0117	0.0002	0.0022	2.40e-06	584.180	43721.73	1640.00	23354.22
option7	10000	0.035	1.1233	0.0059	0.0002	0.0027	3.79e-06	526.172	50425.17	2458.60	28244.71
option7	10000	0.045	1.2134	0.0108	0.0002	0.0050	1.63e-05	1081.23	158458.55	3212.56	76111.96
option7	10000	0.04	1.1650	0.0082	0.0002	0.0031	7.66e-06	775.949	97314.29	2822.58	48027.25
option7	10000	0.055	1.3177	0.0172	0.0003	0.0143	4.34e-05	1805.00	321259.27	4048.61	162458.03
option7	10000	0.05	1.2661	0.0141	0.0002	0.0066	3.08e-05	1431.31	237571.23	3625.91	115669.82

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
option7	10000	0.065	1.1113	0.0027	0.0003	0.0020	4.16e-06	476.896	49693.16	3856.03	32735.29
option7	10000	0.06	1.3682	0.0203	0.0002	0.0131	7.88e-05	2195.21	413182.91	4479.77	217857.79
option7	10000	0.075	1.1551	0.0047	0.0002	0.0042	2.05e-05	787.121	134076.78	4439.04	69294.08
option7	10000	0.07	1.1322	0.0036	0.0003	0.0032	6.24e-06	617.993	84866.70	4143.00	47860.22
option7	10000	0.085	1.2102	0.0079	0.0003	0.0051	2.37e-05	1216.86	283579.99	5077.45	139953.91
option7	10000	0.08	1.1814	0.0062	0.0002	0.0056	3.00e-05	987.050	200428.44	4751.27	100083.03
option7	10000	0.095	1.2751	0.0125	0.0002	0.0105	0.0001	1758.27	506338.80	5775.26	255654.60
option7	10000	0.09	1.2419	0.0100	0.0002	0.0063	2.95e-05	1476.63	385013.00	5420.37	190463.96
option7	16384	0.005	1.1624	0.0199	0.0002	0.0011	4.39e-07	410.658	16673.91	864.852	11033.10
option7	16384	0.015	1.2981	0.0152	0.0003	0.0052	4.59e-06	1372.21	95358.59	2389.33	51854.70
option7	16384	0.01	1.1450	0.0123	0.0002	0.0015	1.56e-06	584.519	39195.47	1515.59	21596.80
option7	16384	0.025	1.2436	0.0121	0.0003	0.0064	1.22e-05	1449.05	193546.10	3436.20	92902.10
option7	16384	0.02	1.1485	0.0083	0.0002	0.0030	4.48e-06	803.647	90925.33	2661.45	44534.84
option7	16384	0.035	1.1162	0.0038	0.0002	0.0020	2.84e-06	673.387	90813.79	4002.57	49109.26
option7	16384	0.03	1.3339	0.0149	0.0003	0.0087	1.44e-05	2214.24	298181.75	4247.70	152387.22
option7	16384	0.045	1.2096	0.0087	0.0002	0.0063	2.75e-05	1546.01	352954.27	5247.15	163108.68
option7	16384	0.04	1.1544	0.0058	0.0002	0.0035	1.07e-05	1049.51	193739.23	4582.52	91132.90
option7	16384	0.055	1.3285	0.0148	0.0002	0.0101	3.22e-05	2793.04	749124.72	6687.61	377053.34
option7	16384	0.05	1.2648	0.0118	0.0002	0.0079	3.84e-05	2135.06	542034.93	5934.41	259547.02
option7	16384	0.065	1.1053	0.0016	0.0002	0.0013	1.85e-06	573.506	76633.50	6283.32	52146.36
option7	16384	0.06	1.3893	0.0176	0.0002	0.0134	6.68e-05	3512.09	959632.00	7453.14	508140.39
option7	16384	0.075	1.1460	0.0030	0.0002	0.0023	4.06e-06	1002.54	234204.50	7215.73	118910.74

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
option7	16384	0.07	1.1219	0.0021	0.0002	0.0022	6.00e-06	761.594	135696.34	6726.11	76729.63
option7	16384	0.085	1.2007	0.0058	0.0003	0.0054	3.25e-05	1653.49	572544.19	8253.69	276159.49
option7	16384	0.08	1.1696	0.0043	0.0002	0.0024	3.20e-06	1299.19	379464.40	7706.55	185464.77
option7	16384	0.095	1.2687	0.0102	0.0002	0.0092	4.27e-05	2528.67	1120010.79	9414.80	560423.52
option7	16384	0.09	1.2316	0.0078	0.0003	0.0073	3.08e-05	2065.18	816782.13	8806.94	401416.64
option8	1024	0.005	1.6035	0.1786	0.0001	0.0033	1.86e-06	49.7563	389.652	74.5680	386.346
option8	1024	0.015	1.3376	0.0700	0.0001	0.0036	3.43e-06	88.3659	948.131	153.895	927.225
option8	1024	0.01	1.4143	0.1019	0.0001	0.0034	1.76e-06	76.4249	740.797	117.004	697.516
option8	1024	0.025	1.2977	0.0461	0.0002	0.0078	1.97e-05	118.172	1397.55	224.116	1375.90
option8	1024	0.02	1.3389	0.0602	0.0001	0.0096	4.08e-05	125.007	1366.18	193.910	1263.63
option8	1024	0.035	1.3330	0.0403	0.0001	0.0089	7.90e-05	185.398	2169.12	298.763	2025.01
option8	1024	0.03	1.3119	0.0427	0.0002	0.0069	2.38e-05	150.898	1776.99	261.127	1693.86
option8	1024	0.045	1.2716	0.0297	0.0002	0.0125	4.27e-05	167.037	2260.72	344.748	2187.23
option8	1024	0.04	1.3570	0.0381	0.0002	0.0133	0.0001	220.962	2545.01	336.649	2348.59
option8	1024	0.055	1.3181	0.0293	0.0001	0.0174	9.60e-05	228.693	3081.60	414.726	2904.62
option8	1024	0.05	1.2939	0.0296	7.00e-05	0.0086	7.50e-05	197.166	2674.31	379.451	2543.54
option8	1024	0.065	1.3676	0.0289	1.00e-04	0.0199	0.0003	293.813	3941.87	485.889	3652.89
option8	1024	0.06	1.3413	0.0290	5.00e-05	0.0230	0.0001	260.416	3480.99	449.719	3259.91
option8	1024	0.075	1.4219	0.0283	0.0001	0.0278	0.0004	363.082	4776.92	559.552	4388.47
option8	1024	0.07	1.3952	0.0285	9.00e-05	0.0206	0.0002	328.356	4346.62	522.775	4010.05
option8	1024	0.085	1.4766	0.0277	0.0001	0.0326	0.0006	435.056	5610.10	634.349	5129.37
option8	1024	0.08	1.4499	0.0281	0.0001	0.0347	0.0004	399.060	5203.99	597.072	4771.19

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
option8	1024	0.095	1.3752	0.0218	0.0001	0.0308	0.0003	338.608	4962.38	637.790	4707.75
option8	1024	0.09	1.3538	0.0216	0.0001	0.0266	0.0003	309.631	4531.01	605.038	4323.09
option8	2048	0.005	1.3578	0.1000	0.0002	0.0016	5.54e-07	82.1932	910.890	126.285	864.916
option8	2048	0.015	1.2442	0.0404	7.00e-05	0.0039	6.81e-06	159.507	2259.79	286.306	2141.50
option8	2048	0.01	1.2932	0.0583	0.0002	0.0045	1.06e-05	138.654	1721.14	213.964	1598.04
option8	2048	0.025	1.2487	0.0279	0.0001	0.0051	1.88e-05	225.070	3451.80	431.298	3329.53
option8	2048	0.02	1.2824	0.0362	1.00e-04	0.0071	3.19e-05	238.129	3328.65	371.470	3040.62
option8	2048	0.035	1.3273	0.0259	7.00e-05	0.0085	5.09e-05	373.423	5613.87	594.982	5199.81
option8	2048	0.03	1.2869	0.0269	9.00e-05	0.0118	3.27e-05	297.241	4522.17	512.323	4266.71
option8	2048	0.045	1.2439	0.0190	5.00e-05	0.0080	5.17e-05	308.075	6462.94	674.499	5597.41
option8	2048	0.04	1.3708	0.0246	9.00e-05	0.0139	6.97e-05	453.974	6598.10	680.203	6061.15
option8	2048	0.055	1.3088	0.0193	8.00e-05	0.0149	0.0001	439.876	8933.54	823.581	7650.00
option8	2048	0.05	1.2753	0.0192	7.00e-05	0.0113	6.15e-05	372.277	7650.91	748.012	6592.64
option8	2048	0.065	1.3788	0.0194	9.00e-05	0.0297	9.67e-05	583.485	11404.78	979.746	9833.30
option8	2048	0.06	1.3435	0.0194	0.0001	0.0177	8.19e-05	510.416	10167.03	900.932	8749.72
option8	2048	0.075	1.4495	0.0193	2.00e-05	0.0371	4.67e-05	735.831	13775.27	1140.84	11941.27
option8	2048	0.07	1.4153	0.0194	1.00e-04	0.0291	0.0001	659.661	12625.47	1060.63	10924.22
option8	2048	0.085	1.5192	0.0193	3.00e-05	0.0521	7.84e-06	894.293	16362.19	1305.37	14257.60
option8	2048	0.08	1.4851	0.0193	5.00e-05	0.0363	0.0004	814.943	15064.67	1223.18	13103.54
option8	2048	0.095	1.3976	0.0150	3.00e-05	0.0124	0.0003	688.802	14427.35	1296.49	12938.26
option8	2048	0.09	1.3706	0.0150	9.00e-05	0.0245	0.0002	624.749	13428.39	1225.12	12015.92
option8	4096	0.005	1.2364	0.0554	0.0001	0.0017	1.24e-06	144.125	2084.50	229.984	1918.41

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
option8	4096	0.015	1.1991	0.0237	9.00e-05	0.0023	6.20e-06	281.223	6041.24	551.848	5019.77
option8	4096	0.01	1.2466	0.0349	9.00e-05	0.0042	6.58e-06	265.245	4155.15	412.515	3825.06
option8	4096	0.025	1.2338	0.0176	0.0001	0.0058	1.73e-05	424.390	9676.42	852.340	8421.17
option8	4096	0.02	1.2660	0.0224	7.00e-05	0.0052	1.98e-05	448.367	9140.42	733.419	7514.40
option8	4096	0.035	1.3451	0.0169	3.00e-05	0.0109	4.82e-06	746.259	15700.62	1205.86	13572.11
option8	4096	0.03	1.2890	0.0173	1.00e-04	0.0114	2.66e-05	579.844	12669.47	1026.29	10961.46
option8	4096	0.045	1.2288	0.0126	0.0001	0.0089	4.09e-05	554.907	20120.43	1332.54	14776.81
option8	4096	0.04	1.4000	0.0165	7.00e-05	0.0197	6.72e-05	920.519	18787.82	1389.36	16322.98
option8	4096	0.055	1.3107	0.0134	4.00e-05	0.0145	6.81e-05	834.671	28376.39	1649.57	21285.27
option8	4096	0.05	1.2684	0.0131	8.00e-05	0.0145	2.61e-05	689.770	24607.21	1487.95	18111.12
option8	4096	0.065	1.3962	0.0139	6.00e-05	0.0191	0.0001	1143.72	36657.28	1984.30	28170.69
option8	4096	0.06	1.3533	0.0137	1.00e-04	0.0166	6.02e-05	985.864	32697.97	1814.95	24720.71
option8	4096	0.075	1.4798	0.0142	6.00e-05	0.0356	3.83e-05	1470.30	44591.82	2329.28	35184.01
option8	4096	0.07	1.4379	0.0141	3.00e-05	0.0314	5.03e-05	1304.45	40697.50	2155.05	31686.25
option8	4096	0.085	1.5618	0.0145	6.00e-05	0.0488	8.39e-05	1811.88	53310.62	2683.82	43112.58
option8	4096	0.08	1.5205	0.0145	3.00e-05	0.0396	1.21e-05	1638.76	49501.68	2504.79	39397.17
option8	4096	0.095	1.4202	0.0113	1.00e-05	0.0156	0.00e+00	1376.24	47483.00	2634.90	38855.90
option8	4096	0.09	1.3883	0.0110	9.00e-05	0.0309	7.49e-05	1238.43	43507.87	2481.92	35314.83
option8	8192	0.005	1.1768	0.0305	0.0001	0.0008	4.67e-07	243.507	5264.12	437.785	4225.92
option8	8192	0.015	1.1763	0.0144	8.00e-05	0.0032	6.77e-06	483.569	17856.99	1082.70	12170.07
option8	8192	0.01	1.2317	0.0208	6.00e-05	0.0041	4.70e-07	498.142	11134.12	815.178	9117.26
option8	8192	0.025	1.2294	0.0115	5.00e-05	0.0065	1.21e-05	786.150	29863.76	1698.66	22017.37

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
option8	8192	0.02	1.2631	0.0145	5.00e-05	0.0075	6.02e-06	831.510	28310.99	1463.50	19447.23
option8	8192	0.035	1.3652	0.0118	0.0001	0.0139	4.76e-05	1469.40	49369.59	2447.68	37965.07
option8	8192	0.03	1.2973	0.0117	5.00e-05	0.0119	3.85e-06	1114.71	39485.07	2065.84	29685.63
option8	8192	0.045	1.2178	0.0089	3.00e-05	0.0030	1.40e-05	984.842	67316.17	2641.38	42091.18
option8	8192	0.04	1.4304	0.0117	0.0001	0.0178	3.51e-05	1840.90	58879.99	2838.90	46298.82
option8	8192	0.055	1.3131	0.0104	7.00e-05	0.0150	3.44e-05	1568.33	100141.67	3305.15	65883.59
option8	8192	0.05	1.2592	0.0098	1.00e-04	0.0135	2.19e-05	1266.40	84674.25	2954.28	54082.56
option8	8192	0.065	1.4107	0.0114	8.00e-05	0.0198	7.88e-05	2217.58	133181.66	4009.80	92446.68
option8	8192	0.06	1.3570	0.0109	7.00e-05	0.0197	3.11e-05	1886.35	116508.39	3639.92	78739.57
option8	8192	0.075	1.5063	0.0122	6.00e-05	0.0289	1.21e-05	2911.27	166146.81	4742.00	120875.11
option8	8192	0.07	1.4544	0.0117	8.00e-05	0.0256	6.55e-05	2559.25	148473.29	4359.72	105611.23
option8	8192	0.085	1.5987	0.0130	1.00e-04	0.0423	6.96e-05	3636.59	202846.70	5494.58	154419.85
option8	8192	0.08	1.5490	0.0126	5.00e-05	0.0390	2.80e-05	3270.92	183133.50	5103.31	136851.87
option8	8192	0.095	1.4392	0.0095	5.00e-05	0.0301	1.63e-05	2720.87	172795.40	5340.07	130757.58
option8	8192	0.09	1.3991	0.0092	2.00e-05	0.0278	5.96e-08	2427.77	157181.97	5002.47	117143.07
option8	10000	0.005	1.1353	0.0257	7.00e-05	0.0004	1.14e-07	276.105	7003.51	515.568	5295.11
option8	10000	0.015	1.1529	0.0121	0.0001	0.0023	4.37e-06	543.774	24556.79	1295.33	15285.90
option8	10000	0.01	1.2125	0.0181	6.00e-05	0.0038	1.75e-06	588.410	15420.20	979.575	11807.63
option8	10000	0.025	1.2113	0.0106	5.00e-05	0.0067	8.92e-06	907.111	44549.84	2042.90	30236.31
option8	10000	0.02	1.2440	0.0134	0.0001	0.0059	9.32e-06	962.293	42983.33	1759.51	26809.39
option8	10000	0.035	1.3539	0.0116	0.0001	0.0134	2.74e-05	1743.99	77465.80	2963.23	56040.47
option8	10000	0.045	1.1995	0.0084	4.00e-05	0.0099	1.32e-05	1112.02	102221.01	3175.76	59009.69

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
option8	10000	0.04	1.4223	0.0118	5.00e-05	0.0124	4.10e-05	2202.98	92930.32	3446.05	69591.83
option8	10000	0.055	1.2844	0.0117	5.00e-05	0.0118	3.67e-05	1771.57	180258.09	3946.44	110944.89
option8	10000	0.05	1.2419	0.0099	6.00e-05	0.0113	2.93e-05	1434.03	136617.37	3556.81	81319.37
option8	10000	0.065	1.3677	0.0160	9.00e-05	0.0162	0.0001	2489.75	290607.13	4745.38	192721.27
option8	10000	0.06	1.3256	0.0137	6.00e-05	0.0124	6.99e-05	2120.83	230070.29	4340.43	146732.85
option8	10000	0.075	1.4468	0.0214	9.00e-05	0.0274	7.64e-05	3245.77	442497.04	5559.98	316694.79
option8	10000	0.07	1.4083	0.0186	6.00e-05	0.0223	0.0002	2866.95	359877.05	5153.10	249265.80
option8	10000	0.085	1.5229	0.0274	9.00e-05	0.0322	0.0003	4034.68	636665.39	6389.35	484104.24
option8	10000	0.08	1.4849	0.0243	5.00e-05	0.0294	0.0002	3635.85	531804.67	5971.97	394041.41
option8	10000	0.095	1.3853	0.0168	8.00e-05	0.0264	2.42e-05	3008.19	460901.94	6274.22	345336.62
option8	10000	0.09	1.3554	0.0145	8.00e-05	0.0248	0.0001	2692.23	380153.49	5915.92	277327.56
option8	16384	0.005	1.1438	0.0170	6.00e-05	0.0007	4.81e-07	395.103	14256.28	851.047	9392.99
option8	16384	0.015	1.1605	0.0089	5.00e-05	0.0033	2.49e-06	814.462	54901.98	2136.45	30210.18
option8	16384	0.01	1.2195	0.0130	9.00e-05	0.0033	1.58e-06	920.204	33589.43	1614.19	22891.38
option8	16384	0.025	1.2257	0.0081	7.00e-05	0.0064	2.30e-06	1436.16	99951.70	3386.98	62304.79
option8	16384	0.02	1.2574	0.0102	5.00e-05	0.0076	2.78e-07	1526.46	95589.01	2913.81	55000.87
option8	16384	0.035	1.3826	0.0092	7.00e-05	0.0140	5.91e-06	2867.42	172478.42	4957.87	119268.24
option8	16384	0.03	1.3005	0.0088	8.00e-05	0.0095	9.93e-06	2122.89	136513.15	4141.81	89627.55
option8	16384	0.045	1.2071	0.0068	5.00e-05	0.0047	1.06e-05	1723.84	230589.81	5236.21	127940.85
option8	16384	0.04	1.4527	0.0096	5.00e-05	0.0194	3.02e-06	3648.89	209589.40	5766.75	151317.00
option8	16384	0.055	1.3126	0.0089	1.00e-04	0.0144	3.26e-05	2919.28	375772.30	6607.61	227004.02
option8	16384	0.05	1.2559	0.0079	5.00e-05	0.0086	3.37e-05	2297.43	300715.79	5892.87	173949.82

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
option8	16384	0.065	1.4217	0.0104	5.00e-05	0.0191	3.71e-05	4273.89	515736.03	8082.22	336644.73
option8	16384	0.06	1.3643	0.0096	4.00e-05	0.0185	4.88e-05	3578.27	441516.82	7319.18	277058.75
option8	16384	0.075	1.5273	0.0114	9.00e-05	0.0279	3.28e-05	5725.04	653376.24	9616.11	455347.45
option8	16384	0.07	1.4733	0.0111	7.00e-05	0.0252	2.30e-05	4992.19	592879.02	8832.55	400871.33
option8	16384	0.085	1.6290	0.0129	6.00e-05	0.0435	8.87e-06	7246.96	827866.40	11197.55	611032.00
option8	16384	0.08	1.5764	0.0123	3.00e-05	0.0321	5.20e-07	6475.93	746231.36	10387.48	536112.36
option8	16384	0.095	1.4530	0.0088	6.00e-05	0.0310	1.49e-05	5336.32	674899.87	10782.11	488607.65
option8	16384	0.09	1.4106	0.0084	0.0001	0.0242	7.05e-05	4718.30	609083.44	10087.19	431489.07
original	1024	0.005	1.1050	0.1322	0.1517	0.0027	1.74e-06	26.4965	222.133	50.2240	282.510
original	1024	0.015	1.1543	0.0445	0.0198	0.0023	1.09e-06	41.7787	327.630	132.678	588.777
original	1024	0.01	1.1560	0.0689	0.0438	0.0026	2.14e-06	37.8902	324.718	95.2948	470.661
original	1024	0.025	1.1585	0.0247	0.0075	0.0022	4.71e-07	45.0226	318.243	200.033	735.421
original	1024	0.02	1.1502	0.0320	0.0105	0.0023	9.51e-07	44.4629	327.111	166.539	671.758
original	1024	0.035	1.1755	0.0169	0.0032	0.0020	1.81e-07	45.5129	274.487	263.450	848.691
original	1024	0.03	1.1661	0.0204	0.0051	0.0021	4.83e-07	46.0464	303.588	232.097	808.543
original	1024	0.045	1.1696	0.0126	0.0021	0.0020	7.16e-08	47.5498	276.091	317.102	922.268
original	1024	0.04	1.1662	0.0146	0.0029	0.0020	1.38e-07	47.6906	290.031	289.337	899.447
original	1024	0.055	1.1804	0.0100	0.0016	0.0020	2.04e-07	46.8741	256.967	371.401	991.228
original	1024	0.05	1.1763	0.0108	0.0020	0.0020	3.80e-08	45.2615	247.721	344.970	930.178
original	1024	0.065	1.1887	0.0084	0.0009	0.0020	0.00e+00	47.6000	248.653	422.352	1065.99
original	1024	0.06	1.1807	0.0092	0.0012	0.0020	9.61e-08	47.5896	255.472	395.883	1033.78
original	1024	0.075	1.2024	0.0068	0.0008	0.0020	4.89e-08	44.5018	196.443	473.199	1055.47

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
original	1024	0.07	1.1975	0.0076	0.0011	0.0020	1.01e-07	46.5968	227.260	448.703	1069.13
original	1024	0.085	1.2016	0.0058	0.0007	0.0020	0.00e+00	44.8243	191.671	516.243	1065.94
original	1024	0.08	1.1967	0.0066	0.0009	0.0020	0.00e+00	48.4319	235.491	492.847	1127.60
original	1024	0.095	1.1972	0.0048	0.0003	0.0020	0.00e+00	42.0389	167.642	555.271	1034.10
original	1024	0.09	1.1966	0.0057	0.0006	0.0020	0.00e+00	48.4011	226.469	534.830	1134.09
original	2048	0.005	1.1382	0.0724	0.0437	0.0013	4.69e-07	42.9628	427.720	105.438	626.576
original	2048	0.01	1.1443	0.0336	0.0112	0.0011	2.42e-07	50.8424	435.927	189.261	919.029
original	2048	0.025	1.1470	0.0121	0.0018	0.0010	3.62e-08	55.3316	379.793	396.166	1438.60
original	2048	0.02	1.1427	0.0158	0.0028	0.0010	4.50e-08	54.9089	392.418	331.005	1322.66
original	2048	0.035	1.1665	0.0083	0.0009	0.0010	0.00e+00	54.6910	313.144	522.890	1671.06
original	2048	0.03	1.1566	0.0101	0.0013	0.0010	2.80e-08	56.3059	363.597	460.437	1603.92
original	2048	0.045	1.1644	0.0062	0.0007	0.0010	0.00e+00	56.4974	312.368	631.386	1829.77
original	2048	0.04	1.1628	0.0071	0.0007	0.0010	0.00e+00	56.9438	329.964	577.006	1759.25
original	2048	0.055	1.1741	0.0049	0.0004	0.0010	0.00e+00	55.4172	286.263	738.831	1956.04
original	2048	0.05	1.1704	0.0054	0.0004	0.0010	0.00e+00	53.5852	279.266	686.494	1844.32
original	2048	0.065	1.1856	0.0042	0.0002	0.0010	0.00e+00	55.9318	273.074	842.530	2103.30
original	2048	0.06	1.1782	0.0045	0.0003	0.0010	0.00e+00	56.1970	283.677	790.099	2044.06
original	2048	0.075	1.1995	0.0034	0.0002	0.0010	0.00e+00	51.6364	212.770	944.066	2082.13
original	2048	0.07	1.1941	0.0038	0.0002	0.0010	0.00e+00	54.6452	254.898	894.887	2131.09
original	2048	0.085	1.1984	0.0029	0.0001	0.0010	0.00e+00	51.8247	205.402	1029.75	2131.50
original	2048	0.08	1.1937	0.0033	0.0002	0.0010	0.00e+00	56.4711	260.175	983.206	2250.00
original	2048	0.095	1.1930	0.0024	0.0002	0.0010	0.00e+00	48.4751	181.133	1106.68	2048.57

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
original	2048	0.09	1.1939	0.0028	0.0002	0.0010	0.00e+00	56.0761	249.360	1067.19	2254.27
original	4096	0.005	1.1118	0.0356	0.0105	0.0006	4.48e-08	57.7499	573.752	206.714	1232.85
original	4096	0.015	1.1293	0.0107	0.0012	0.0005	3.75e-09	64.9602	489.859	519.721	2269.10
original	4096	0.01	1.1337	0.0168	0.0026	0.0005	8.03e-09	63.2577	522.991	375.158	1834.87
original	4096	0.025	1.1378	0.0061	0.0004	0.0005	0.00e+00	65.6283	430.585	786.032	2890.08
original	4096	0.02	1.1404	0.0078	0.0006	0.0005	3.67e-09	65.9270	449.731	660.703	2629.22
original	4096	0.035	1.1611	0.0042	0.0002	0.0005	0.00e+00	63.8648	342.005	1040.99	3349.65
original	4096	0.03	1.1528	0.0050	0.0002	0.0005	0.00e+00	66.0836	400.634	917.911	3162.74
original	4096	0.045	1.1571	0.0031	0.0002	0.0005	0.00e+00	65.6225	340.201	1254.89	3623.73
original	4096	0.04	1.1605	0.0036	0.0002	0.0005	0.00e+00	66.1168	356.243	1151.69	3549.31
original	4096	0.055	1.1690	0.0025	1.00e-04	0.0005	0.00e+00	63.9127	307.872	1471.32	3906.51
original	4096	0.05	1.1672	0.0027	0.0001	0.0005	0.00e+00	61.8414	294.065	1369.21	3654.30
original	4096	0.065	1.1816	0.0021	0.0002	0.0005	0.00e+00	64.1954	292.705	1679.37	4177.49
original	4096	0.06	1.1773	0.0022	6.00e-05	0.0005	0.00e+00	64.8103	310.139	1578.97	4042.92
original	4096	0.075	1.1962	0.0017	6.00e-05	0.0005	0.00e+00	58.9832	224.678	1883.06	4164.16
original	4096	0.07	1.1913	0.0019	4.00e-05	0.0005	0.00e+00	62.2086	264.233	1785.54	4185.68
original	4096	0.085	1.1962	0.0014	2.00e-05	0.0005	0.00e+00	58.9172	218.875	2055.75	4233.07
original	4096	0.08	1.1921	0.0016	5.00e-05	0.0005	0.00e+00	64.5123	281.628	1963.70	4461.44
original	4096	0.095	1.1941	0.0012	3.00e-05	0.0005	0.00e+00	54.9282	188.926	2215.33	4067.71
original	4096	0.09	1.1930	0.0014	5.00e-05	0.0005	0.00e+00	63.8352	262.564	2132.83	4494.39
original	8192	0.005	1.1107	0.0175	0.0029	0.0003	3.50e-09	72.1152	696.211	413.211	2415.58
original	8192	0.015	1.1309	0.0054	0.0002	0.0002	0.00e+00	76.4240	530.826	1040.95	4533.80

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
original	8192	0.01	1.1260	0.0083	0.0006	0.0003	2.05e-09	75.6003	598.368	745.274	3625.71
original	8192	0.025	1.1378	0.0030	0.0002	0.0002	0.00e+00	76.0683	454.882	1572.05	5720.27
original	8192	0.02	1.1387	0.0039	0.0002	0.0002	0.00e+00	76.8506	483.077	1319.38	5221.44
original	8192	0.035	1.1614	0.0021	6.00e-05	0.0002	0.00e+00	73.0627	362.984	2082.39	6674.11
original	8192	0.03	1.1516	0.0025	8.00e-05	0.0002	0.00e+00	76.0349	425.699	1833.91	6319.57
original	8192	0.045	1.1566	0.0015	4.00e-05	0.0002	0.00e+00	74.6634	357.873	2508.68	7237.68
original	8192	0.04	1.1604	0.0018	4.00e-05	0.0002	0.00e+00	75.5797	382.925	2303.31	7051.22
original	8192	0.055	1.1692	0.0012	2.00e-05	0.0002	0.00e+00	72.5200	327.603	2943.00	7762.73
original	8192	0.05	1.1657	0.0013	2.00e-05	0.0002	0.00e+00	70.2624	312.183	2734.91	7311.02
original	8192	0.065	1.1819	0.0010	2.00e-05	0.0002	0.00e+00	72.5536	308.007	3359.38	8382.75
original	8192	0.06	1.1761	0.0011	2.00e-05	0.0002	0.00e+00	73.3537	324.726	3154.77	8121.79
original	8192	0.075	1.1964	0.0008	1.00e-05	0.0002	0.00e+00	66.1714	233.847	3766.60	8304.38
original	8192	0.07	1.1903	0.0009	2.00e-05	0.0002	0.00e+00	70.0960	276.233	3568.12	8422.80
original	8192	0.085	1.1966	0.0007	0.00e+00	0.00e+00	0.00e+00	65.9733	227.380	4112.66	8450.22
original	8192	0.08	1.1913	0.0008	2.00e-05	0.0002	0.00e+00	72.5228	293.322	3924.90	8889.74
original	8192	0.095	1.1948	0.0006	1.00e-05	0.0002	0.00e+00	61.3739	196.036	4433.45	8125.21
original	8192	0.09	1.1923	0.0007	1.00e-05	0.0002	0.00e+00	71.5847	273.622	4263.28	8943.19
original	10000	0.0025	1.1218	0.0291	0.0063	0.0002	0.00e+00	69.7652	754.322	282.738	1844.94
original	10000	0.005	1.1062	0.0143	0.0016	0.0002	4.82e-10	75.7938	715.889	502.384	2948.02
original	10000	0.015	1.1329	0.0043	0.0002	0.0002	0.00e+00	79.8496	545.854	1272.93	5491.09
original	10000	0.01	1.1239	0.0067	0.0005	0.0002	1.75e-10	78.0764	642.832	908.002	4383.22
original	10000	0.025	1.1368	0.0024	7.00e-05	0.0002	0.00e+00	79.1491	467.222	1917.40	6965.73

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
original	10000	0.02	1.1389	0.0032	6.00e-05	0.0002	0.00e+00	80.0937	498.735	1610.90	6347.43
original	10000	0.035	1.1614	0.0017	6.00e-05	0.0002	0.00e+00	75.9089	371.485	2542.06	8073.00
original	10000	0.03	1.1514	0.0020	7.00e-05	0.0002	0.00e+00	79.0422	433.780	2238.22	7657.23
original	10000	0.045	1.1572	0.0013	0.00e+00	0.00e+00	0.00e+00	77.2507	361.035	3063.76	8786.93
original	10000	0.04	1.1600	0.0015	2.00e-05	0.0002	0.00e+00	78.1251	382.596	2810.55	8624.29
original	10000	0.055	1.1691	0.0010	3.00e-05	0.0002	0.00e+00	75.0377	329.137	3592.20	9480.77
original	10000	0.05	1.1652	0.0011	1.00e-05	0.0002	0.00e+00	72.6491	314.443	3337.17	8972.60
original	10000	0.065	1.1819	0.0008	0.00e+00	0.00e+00	0.00e+00	74.9685	315.100	4101.05	10202.17
original	10000	0.06	1.1760	0.0009	1.00e-05	0.0002	0.00e+00	75.8672	332.457	3850.67	9951.69
original	10000	0.075	1.1959	0.0007	0.00e+00	0.00e+00	0.00e+00	68.3592	238.471	4596.08	10143.23
original	10000	0.07	1.1903	0.0008	2.00e-05	0.0002	0.00e+00	72.6042	282.284	4355.68	10299.68
original	10000	0.085	1.1962	0.0006	0.00e+00	0.00e+00	0.00e+00	68.0205	229.276	5018.77	10379.82
original	10000	0.08	1.1906	0.0007	2.00e-05	0.0002	0.00e+00	74.9409	296.051	4788.54	10796.85
original	10000	0.095	1.1952	0.0005	0.00e+00	0.00e+00	0.00e+00	63.2705	197.691	5413.49	9950.40
original	10000	0.09	1.1920	0.0006	0.00e+00	0.00e+00	0.00e+00	73.8501	277.275	5202.75	10924.74
original	16384	0.005	1.1084	0.0087	0.0008	0.0001	1.86e-10	86.4073	786.140	824.757	4806.86
original	16384	0.015	1.1320	0.0027	5.00e-05	0.0001	0.00e+00	88.0784	575.363	2083.90	9042.61
original	16384	0.01	1.1227	0.0041	9.00e-05	0.0001	0.00e+00	88.1581	647.420	1486.12	7205.12
original	16384	0.025	1.1372	0.0015	2.00e-05	0.0001	0.00e+00	86.6259	487.064	3142.60	11389.24
original	16384	0.02	1.1380	0.0019	9.00e-05	0.0001	0.00e+00	87.9078	515.943	2637.27	10353.22
original	16384	0.035	1.1612	0.0010	0.00e+00	0.00e+00	0.00e+00	82.3537	381.164	4164.29	13303.37
original	16384	0.03	1.1511	0.0012	1.00e-05	0.0001	0.00e+00	86.1119	446.444	3666.28	12575.85

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
original	16384	0.045	1.1566	0.0008	0.00e+00	0.00e+00	0.00e+00	83.7631	374.154	5017.27	14379.04
original	16384	0.04	1.1596	0.0009	0.00e+00	0.00e+00	0.00e+00	84.9563	395.202	4603.36	14104.93
original	16384	0.055	1.1690	0.0006	0.00e+00	0.00e+00	0.00e+00	81.1119	336.262	5885.08	15502.42
original	16384	0.05	1.1648	0.0007	0.00e+00	0.00e+00	0.00e+00	78.7100	323.176	5465.85	14624.55
original	16384	0.065	1.1819	0.0005	1.00e-05	0.0001	0.00e+00	81.0562	319.623	6718.77	16718.58
original	16384	0.06	1.1756	0.0006	0.00e+00	0.00e+00	0.00e+00	82.0088	337.725	6307.07	16320.83
original	16384	0.075	1.1965	0.0004	0.00e+00	0.00e+00	0.00e+00	73.5001	243.094	7533.90	16646.17
original	16384	0.07	1.1899	0.0005	2.00e-05	0.0001	0.00e+00	78.2140	287.004	7133.61	16907.37
original	16384	0.085	1.1965	0.0004	0.00e+00	0.00e+00	0.00e+00	73.0903	232.980	8224.80	16843.76
original	16384	0.08	1.1909	0.0004	0.00e+00	0.00e+00	0.00e+00	80.7208	301.900	7846.89	17829.83
original	16384	0.095	1.1950	0.0003	0.00e+00	0.00e+00	0.00e+00	67.8489	199.435	8868.07	16243.08
original	16384	0.09	1.1920	0.0003	0.00e+00	0.00e+00	0.00e+00	79.5108	283.617	8524.22	17895.49
sugimoto	1024	0.005	1.4186	0.1691	0.0026	0.0028	2.22e-06	39.1511	340.518	65.9321	366.126
sugimoto	1024	0.015	1.2422	0.0683	0.0035	0.0045	9.58e-06	73.7691	899.146	142.818	906.121
sugimoto	1024	0.01	1.2826	0.0937	0.0030	0.0040	6.34e-06	57.9623	618.563	106.039	642.112
sugimoto	1024	0.025	1.1977	0.0419	0.0033	0.0054	1.51e-05	90.5187	1282.24	206.733	1252.08
sugimoto	1024	0.02	1.2256	0.0518	0.0030	0.0048	1.05e-05	83.6415	1085.46	177.412	1088.70
sugimoto	1024	0.035	1.1954	0.0333	0.0035	0.0065	2.46e-05	107.408	1814.21	267.788	1678.36
sugimoto	1024	0.03	1.1954	0.0369	0.0035	0.0063	1.97e-05	99.7269	1547.04	237.791	1466.36
sugimoto	1024	0.045	1.1815	0.0259	0.0032	0.0068	3.21e-05	115.059	2110.90	320.170	1905.49
sugimoto	1024	0.04	1.1894	0.0282	0.0034	0.0063	2.31e-05	109.219	1888.53	294.958	1738.83
sugimoto	1024	0.055	1.1806	0.0228	0.0036	0.0075	3.48e-05	126.330	2642.42	371.265	2268.06

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
sugimoto	1024	0.05	1.1737	0.0240	0.0036	0.0066	3.05e-05	118.212	2340.58	344.052	2065.88
sugimoto	1024	0.065	1.1855	0.0210	0.0035	0.0091	5.26e-05	138.552	3242.30	421.010	2655.70
sugimoto	1024	0.06	1.1843	0.0218	0.0035	0.0077	4.18e-05	133.177	2939.56	396.897	2456.62
sugimoto	1024	0.075	1.1842	0.0176	0.0038	0.0082	4.44e-05	137.198	3294.91	465.795	2740.36
sugimoto	1024	0.07	1.1898	0.0194	0.0040	0.0087	5.98e-05	140.759	3375.95	445.578	2741.68
sugimoto	1024	0.085	1.1890	0.0167	0.0034	0.0094	6.27e-05	148.462	3890.29	510.612	3084.43
sugimoto	1024	0.08	1.1759	0.0156	0.0032	0.0087	5.38e-05	130.667	3110.55	484.112	2659.51
sugimoto	1024	0.095	1.1902	0.0161	0.0032	0.0109	9.22e-05	156.423	4492.70	551.794	3469.33
sugimoto	1024	0.09	1.1763	0.0150	0.0030	0.0089	6.04e-05	138.905	3663.50	525.569	3014.06
sugimoto	2048	0.005	1.2511	0.0948	0.0031	0.0019	1.37e-06	64.1224	786.930	116.294	821.358
sugimoto	2048	0.01	1.1946	0.0509	0.0035	0.0025	2.96e-06	93.8144	1379.41	197.549	1396.23
sugimoto	2048	0.025	1.1462	0.0231	0.0036	0.0035	8.76e-06	137.913	3064.66	395.707	2762.71
sugimoto	2048	0.02	1.1612	0.0275	0.0033	0.0035	6.58e-06	125.905	2443.44	336.219	2317.63
sugimoto	2048	0.035	1.1499	0.0175	0.0032	0.0041	1.21e-05	155.159	4058.74	515.257	3518.80
sugimoto	2048	0.03	1.1488	0.0197	0.0036	0.0041	1.05e-05	147.378	3529.70	457.127	3133.21
sugimoto	2048	0.045	1.1443	0.0140	0.0034	0.0043	1.50e-05	167.484	4913.15	620.292	4140.16
sugimoto	2048	0.04	1.1448	0.0147	0.0032	0.0040	1.15e-05	156.506	4178.04	567.883	3633.28
sugimoto	2048	0.055	1.1406	0.0118	0.0033	0.0050	1.92e-05	177.823	5795.99	717.488	4690.61
sugimoto	2048	0.05	1.1332	0.0122	0.0036	0.0047	1.75e-05	164.708	4937.01	664.395	4193.47
sugimoto	2048	0.065	1.1486	0.0109	0.0035	0.0057	2.63e-05	194.202	7118.73	815.910	5507.46
sugimoto	2048	0.06	1.1463	0.0113	0.0036	0.0052	1.99e-05	187.985	6513.05	768.456	5113.19
sugimoto	2048	0.075	1.1487	0.0091	0.0032	0.0052	2.66e-05	189.407	7062.50	903.891	5623.09

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
sugimoto	2048	0.07	1.1517	0.0102	0.0030	0.0052	2.19e-05	196.156	7431.71	862.874	5732.93
sugimoto	2048	0.085	1.1579	0.0092	0.0035	0.0062	3.17e-05	212.388	9143.07	994.623	6825.26
sugimoto	2048	0.08	1.1442	0.0083	0.0034	0.0048	2.06e-05	181.903	6714.65	942.211	5610.46
sugimoto	2048	0.095	1.1464	0.0081	0.0035	0.0059	2.62e-05	214.322	9440.68	1063.10	7029.42
sugimoto	2048	0.09	1.1436	0.0074	0.0030	0.0052	2.24e-05	186.438	7256.64	1022.02	5912.25
sugimoto	4096	0.005	1.1246	0.0512	0.0032	0.0012	6.36e-07	104.295	1722.62	209.068	1775.79
sugimoto	4096	0.015	1.1084	0.0189	0.0033	0.0020	3.00e-06	168.990	4474.38	509.922	4011.72
sugimoto	4096	0.01	1.1411	0.0273	0.0033	0.0018	2.10e-06	142.469	3101.32	377.445	2998.83
sugimoto	4096	0.025	1.0989	0.0119	0.0033	0.0021	2.92e-06	192.974	6523.39	758.947	5696.13
sugimoto	4096	0.02	1.1228	0.0145	0.0035	0.0019	2.94e-06	181.178	5409.19	650.245	4870.90
sugimoto	4096	0.035	1.1090	0.0088	0.0034	0.0024	5.60e-06	212.773	8351.14	993.924	7103.98
sugimoto	4096	0.03	1.1162	0.0100	0.0036	0.0026	5.05e-06	205.667	7467.97	888.486	6386.36
sugimoto	4096	0.045	1.1079	0.0069	0.0030	0.0025	6.98e-06	226.034	9754.44	1201.22	8171.35
sugimoto	4096	0.04	1.1179	0.0078	0.0032	0.0026	7.26e-06	218.641	9044.38	1109.18	7683.02
sugimoto	4096	0.055	1.1090	0.0060	0.0030	0.0030	8.96e-06	243.474	11956.94	1395.47	9533.01
sugimoto	4096	0.05	1.1073	0.0060	0.0032	0.0028	6.73e-06	221.677	9754.20	1298.69	8308.16
sugimoto	4096	0.065	1.1198	0.0056	0.0035	0.0033	1.08e-05	268.269	15228.41	1591.08	11411.52
sugimoto	4096	0.06	1.1210	0.0057	0.0034	0.0031	9.23e-06	256.825	13437.45	1503.10	10365.71
sugimoto	4096	0.075	1.1198	0.0045	0.0032	0.0028	7.98e-06	252.051	13690.03	1762.42	11114.34
sugimoto	4096	0.07	1.1263	0.0050	0.0033	0.0035	1.24e-05	262.547	14652.43	1687.72	11258.17
sugimoto	4096	0.085	1.1291	0.0045	0.0033	0.0036	1.20e-05	282.090	17718.55	1940.03	13257.96
sugimoto	4096	0.08	1.1217	0.0039	0.0032	0.0027	7.82e-06	235.832	12317.77	1847.55	10722.89

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
sugimoto	4096	0.095	1.1302	0.0041	0.0035	0.0037	1.44e-05	289.387	19098.36	2096.50	14124.44
sugimoto	4096	0.09	1.1233	0.0036	0.0030	0.0027	7.76e-06	245.151	13514.28	2007.96	11456.53
sugimoto	8192	0.005	1.1015	0.0266	0.0030	0.0008	3.93e-07	158.954	3798.59	409.620	3694.76
sugimoto	8192	0.015	1.0901	0.0097	0.0032	0.0011	1.01e-06	236.103	9513.70	1003.09	8209.33
sugimoto	8192	0.01	1.1069	0.0141	0.0030	0.0011	1.01e-06	205.222	6764.43	732.355	6178.04
sugimoto	8192	0.025	1.0839	0.0058	0.0036	0.0013	1.65e-06	260.869	12680.86	1497.26	11146.77
sugimoto	8192	0.02	1.0998	0.0074	0.0030	0.0013	1.61e-06	253.583	11510.08	1274.02	10017.77
sugimoto	8192	0.035	1.0939	0.0043	0.0031	0.0014	1.85e-06	281.028	15599.45	1961.07	13704.50
sugimoto	8192	0.03	1.0958	0.0050	0.0031	0.0013	2.05e-06	278.565	14881.38	1744.74	12743.85
sugimoto	8192	0.045	1.0947	0.0034	0.0031	0.0016	2.48e-06	298.272	18396.91	2374.03	15948.09
sugimoto	8192	0.04	1.0993	0.0037	0.0030	0.0017	2.57e-06	285.703	16370.07	2181.54	14570.02
sugimoto	8192	0.055	1.0948	0.0028	0.0034	0.0017	3.22e-06	316.530	21240.26	2755.37	17901.79
sugimoto	8192	0.05	1.0899	0.0029	0.0031	0.0014	1.71e-06	286.026	17308.64	2556.69	15808.33
sugimoto	8192	0.065	1.1046	0.0026	0.0033	0.0018	3.18e-06	347.532	26978.81	3139.32	21291.55
sugimoto	8192	0.06	1.1047	0.0028	0.0034	0.0016	2.63e-06	340.167	25404.67	2962.91	20240.47
sugimoto	8192	0.075	1.1076	0.0021	0.0035	0.0017	3.42e-06	326.173	23885.32	3486.54	20846.62
sugimoto	8192	0.07	1.1099	0.0024	0.0030	0.0017	3.13e-06	340.494	25989.46	3326.64	21312.71
sugimoto	8192	0.085	1.1155	0.0021	0.0033	0.0020	4.56e-06	359.962	30405.87	3833.43	24352.70
sugimoto	8192	0.08	1.1072	0.0018	0.0034	0.0015	3.21e-06	293.774	19397.27	3647.46	19237.70
sugimoto	8192	0.095	1.1164	0.0019	0.0034	0.0023	6.55e-06	367.879	32297.86	4142.02	25834.00
sugimoto	8192	0.09	1.1083	0.0016	0.0032	0.0017	3.25e-06	305.122	21215.61	3962.57	20484.98
sugimoto	10000	0.005	1.0942	0.0223	0.0033	0.0008	3.89e-07	177.314	4821.45	496.676	4622.28

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
sugimoto	1000	0.015	1.0884	0.0081	0.0034	0.0012	1.12e-06	263.291	12155.07	1222.62	10252.12
sugimoto	1000	0.01	1.1024	0.0119	0.0035	0.0009	6.25e-07	232.005	8800.79	890.318	7771.96
sugimoto	1000	0.025	1.0799	0.0048	0.0032	0.0013	1.38e-06	281.936	15134.35	1820.98	13554.14
sugimoto	1000	0.02	1.0951	0.0061	0.0034	0.0013	1.38e-06	275.975	14016.88	1548.57	12221.08
sugimoto	1000	0.035	1.0905	0.0035	0.0031	0.0012	1.67e-06	305.829	18705.58	2386.44	16699.90
sugimoto	1000	0.03	1.0922	0.0041	0.0031	0.0012	1.48e-06	303.371	18121.79	2122.86	15519.53
sugimoto	1000	0.045	1.0910	0.0027	0.0035	0.0013	1.93e-06	318.338	20955.29	2888.31	18924.40
sugimoto	1000	0.04	1.0947	0.0030	0.0032	0.0012	1.54e-06	306.943	19293.97	2651.98	17648.82
sugimoto	1000	0.055	1.0914	0.0023	0.0033	0.0013	2.09e-06	338.951	25081.59	3353.22	21770.64
sugimoto	1000	0.05	1.0868	0.0023	0.0037	0.0012	1.60e-06	310.430	20427.38	3112.30	19237.28
sugimoto	1000	0.065	1.1008	0.0021	0.0034	0.0017	2.87e-06	371.292	31086.23	3819.24	25348.87
sugimoto	1000	0.06	1.1004	0.0022	0.0034	0.0015	2.67e-06	363.423	29244.49	3602.64	24012.48
sugimoto	1000	0.075	1.1047	0.0017	0.0034	0.0013	1.94e-06	345.865	27094.98	4245.16	24687.70
sugimoto	1000	0.07	1.1068	0.0019	0.0034	0.0016	2.95e-06	365.890	30496.53	4049.63	25460.47
sugimoto	1000	0.085	1.1123	0.0016	0.0033	0.0017	3.90e-06	385.249	34876.35	4666.38	28995.91
sugimoto	1000	0.08	1.1042	0.0014	0.0034	0.0013	2.12e-06	313.202	22130.55	4440.32	22854.42
sugimoto	1000	0.095	1.1132	0.0015	0.0031	0.0016	3.52e-06	390.001	36530.43	5041.88	30336.11
sugimoto	1000	0.09	1.1054	0.0013	0.0030	0.0012	1.65e-06	323.070	23892.67	4824.21	24209.54
sugimoto	16384	0.005	1.0836	0.0137	0.0032	0.0005	2.11e-07	229.091	8209.51	806.036	7630.01
sugimoto	16384	0.015	1.0774	0.0048	0.0031	0.0008	5.15e-07	319.838	18568.32	1982.98	16329.10
sugimoto	16384	0.01	1.0870	0.0072	0.0032	0.0007	3.86e-07	287.168	14088.28	1438.50	12599.44
sugimoto	16384	0.025	1.0716	0.0028	0.0036	0.0008	6.96e-07	336.799	22089.62	2960.67	21092.88

Table A.1: First Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
sugimoto	16384	0.02	1.0837	0.0035	0.0032	0.0008	5.90e-07	331.273	20681.44	2510.94	19057.06
sugimoto	16384	0.035	1.0828	0.0020	0.0036	0.0008	7.45e-07	357.557	25712.62	3882.36	25455.41
sugimoto	16384	0.03	1.0814	0.0024	0.0033	0.0007	4.95e-07	357.416	25558.74	3443.69	23908.52
sugimoto	16384	0.045	1.0844	0.0016	0.0030	0.0008	7.46e-07	374.382	29374.05	4703.43	29208.53
sugimoto	16384	0.04	1.0862	0.0017	0.0032	0.0009	8.33e-07	357.218	26103.84	4311.59	26837.02
sugimoto	16384	0.055	1.0848	0.0013	0.0034	0.0008	9.20e-07	396.411	34155.78	5460.64	32807.09
sugimoto	16384	0.05	1.0787	0.0013	0.0031	0.0008	7.22e-07	357.423	26615.74	5061.40	28864.28
sugimoto	16384	0.065	1.0943	0.0012	0.0036	0.0010	1.15e-06	433.698	42759.36	6220.55	38374.20
sugimoto	16384	0.06	1.0926	0.0013	0.0037	0.0010	1.07e-06	424.544	40144.50	5861.03	36334.80
sugimoto	16384	0.075	1.0989	0.0009	0.0032	0.0009	9.55e-07	398.483	35601.76	6918.53	36957.54
sugimoto	16384	0.07	1.0996	0.0011	0.0036	0.0010	1.47e-06	425.805	41576.27	6591.69	38859.16
sugimoto	16384	0.085	1.1061	0.0009	0.0035	0.0010	1.32e-06	443.407	45460.05	7602.81	42585.86
sugimoto	16384	0.08	1.0981	0.0008	0.0031	0.0009	8.89e-07	355.237	27447.79	7235.41	34193.11
sugimoto	16384	0.095	1.1071	0.0008	0.0038	0.0011	1.68e-06	450.904	48652.69	8215.37	45144.70
sugimoto	16384	0.09	1.0994	0.0007	0.0034	0.0009	1.09e-06	367.630	30034.22	7861.46	36175.61

Table A.2: Second Experiment Results

biconfbpi	1024	0.01	1.2239	0.1081	0.0012	0.0036	4.39e-06	52.8463	354.445	101.243	739.838
biconfbpi	1024	0.02	1.1750	0.0564	0.0012	0.0042	8.71e-06	69.5401	545.522	170.165	1183.55
biconfbpi	1024	0.03	1.1485	0.0379	0.0015	0.0053	1.50e-05	76.2505	667.704	228.596	1502.42
biconfbpi	1024	0.04	1.1418	0.0302	0.0017	0.0056	1.65e-05	84.2791	819.556	283.268	1860.10
biconfbpi	1024	0.05	1.1285	0.0238	0.0017	0.0068	2.76e-05	85.8144	882.443	330.939	2049.02
biconfbpi	1024	0.06	1.1182	0.0187	0.0016	0.0057	2.19e-05	85.6385	896.672	374.908	2097.78
biconfbpi	1024	0.07	1.1195	0.0161	0.0022	0.0063	2.46e-05	88.1561	971.258	419.445	2262.48
biconfbpi	1024	0.08	1.1150	0.0133	0.0016	0.0065	2.71e-05	84.9363	938.385	459.170	2258.32
biconfbpi	1024	0.09	1.1113	0.0117	0.0017	0.0070	2.95e-05	87.3311	987.822	496.651	2346.89
biconfbpi	2048	0.01	1.1521	0.0549	0.0014	0.0025	3.41e-06	77.9998	701.956	190.599	1502.59
biconfbpi	2048	0.02	1.1243	0.0298	0.0020	0.0032	6.06e-06	97.5753	1091.41	325.636	2503.07
biconfbpi	2048	0.03	1.1041	0.0190	0.0019	0.0031	5.53e-06	102.176	1244.89	439.551	3009.86
biconfbpi	2048	0.04	1.1045	0.0144	0.0019	0.0035	9.37e-06	110.137	1477.07	548.052	3548.61
biconfbpi	2048	0.05	1.0927	0.0109	0.0020	0.0036	9.35e-06	109.239	1473.54	640.886	3739.58
biconfbpi	2048	0.06	1.0856	0.0086	0.0021	0.0034	7.27e-06	107.900	1487.21	727.987	3857.99
biconfbpi	2048	0.07	1.0886	0.0074	0.0021	0.0037	8.31e-06	110.635	1615.11	815.774	4180.66
biconfbpi	2048	0.08	1.0846	0.0059	0.0018	0.0034	8.44e-06	103.941	1437.74	893.311	3973.23
biconfbpi	2048	0.09	1.0831	0.0053	0.0020	0.0033	6.48e-06	107.099	1547.78	968.198	4207.20
biconfbpi	4096	0.01	1.1104	0.0293	0.0017	0.0014	1.22e-06	110.605	1409.12	367.432	3207.98
biconfbpi	4096	0.02	1.0910	0.0143	0.0020	0.0018	2.07e-06	128.439	1939.67	632.034	4790.06
biconfbpi	4096	0.03	1.0753	0.0087	0.0018	0.0020	2.98e-06	129.892	2057.42	856.155	5524.61
biconfbpi	4096	0.04	1.0771	0.0066	0.0020	0.0020	2.78e-06	137.883	2383.93	1068.89	6454.22
biconfbpi	4096	0.05	1.0694	0.0048	0.0021	0.0017	2.02e-06	134.019	2283.25	1254.51	6663.85

Table A.2: Second Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
biconfbpi	4096	0.06	1.0664	0.0038	0.0021	0.0020	3.77e-06	132.961	2279.60	1430.20	6890.13
biconfbpi	4096	0.07	1.0693	0.0033	0.0025	0.0020	2.77e-06	134.815	2460.50	1602.75	7387.52
biconfbpi	4096	0.08	1.0663	0.0025	0.0018	0.0018	1.74e-06	123.566	1973.62	1756.49	6747.65
biconfbpi	4096	0.09	1.0660	0.0023	0.0020	0.0018	2.91e-06	127.725	2204.91	1905.85	7221.20
biconfbpi	8192	0.01	1.0797	0.0136	0.0020	0.0008	4.52e-07	145.933	2424.64	714.583	5973.00
biconfbpi	8192	0.02	1.0674	0.0065	0.0019	0.0010	5.12e-07	162.075	3118.50	1236.77	8689.54
biconfbpi	8192	0.03	1.0579	0.0039	0.0021	0.0011	1.00e-06	160.464	3121.44	1684.55	9888.40
biconfbpi	8192	0.04	1.0604	0.0029	0.0021	0.0011	1.01e-06	168.528	3515.87	2104.67	11380.65
biconfbpi	8192	0.05	1.0555	0.0021	0.0020	0.0010	6.83e-07	160.062	3138.10	2476.41	11544.79
biconfbpi	8192	0.06	1.0527	0.0017	0.0021	0.0011	8.76e-07	157.429	3122.75	2823.81	11934.46
biconfbpi	8192	0.07	1.0568	0.0014	0.0024	0.0010	8.04e-07	158.878	3215.59	3167.78	12463.60
biconfbpi	8192	0.08	1.0556	0.0011	0.0019	0.0010	7.75e-07	143.553	2541.14	3477.72	11635.51
biconfbpi	8192	0.09	1.0561	0.0010	0.0023	0.0010	8.49e-07	149.085	2896.07	3776.13	12470.03
biconfbpi	10000	0.01	1.0732	0.0110	0.0023	0.0008	3.57e-07	157.448	2868.55	866.990	7157.36
biconfbpi	10000	0.02	1.0623	0.0052	0.0024	0.0009	5.42e-07	172.635	3589.63	1502.46	10324.79
biconfbpi	10000	0.03	1.0536	0.0031	0.0020	0.0008	5.74e-07	169.251	3453.29	2048.07	11650.55
biconfbpi	10000	0.04	1.0564	0.0023	0.0024	0.0008	5.04e-07	178.063	3982.30	2559.52	13547.13
biconfbpi	10000	0.05	1.0528	0.0017	0.0019	0.0009	5.45e-07	167.988	3452.53	3015.20	13600.07
biconfbpi	10000	0.06	1.0497	0.0013	0.0024	0.0008	5.07e-07	163.916	3319.17	3437.18	13977.22
biconfbpi	10000	0.07	1.0544	0.0011	0.0021	0.0009	1.02e-06	166.188	3525.12	3858.37	14709.92
biconfbpi	10000	0.08	1.0538	0.0008	0.0020	0.0008	7.20e-07	150.077	2731.18	4238.16	13715.97
biconfbpi	10000	0.09	1.0537	0.0008	0.0020	0.0009	6.41e-07	154.745	3027.24	4599.26	14408.56

Table A.2: Second Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
biconf7bpi	16384	0.01	1.0594	0.0063	0.0022	0.0005	1.90e-07	185.176	3983.26	1402.26	11106.61
biconf7bpi	16384	0.02	1.0526	0.0029	0.0024	0.0005	2.26e-07	198.853	4645.58	2439.33	15605.50
biconf7bpi	16384	0.03	1.0458	0.0017	0.0020	0.0005	2.11e-07	191.257	4246.07	3330.72	17566.22
biconf7bpi	16384	0.04	1.0498	0.0013	0.0027	0.0006	3.27e-07	199.076	4731.42	4167.42	20137.33
biconf7bpi	16384	0.05	1.0469	0.0009	0.0020	0.0005	2.38e-07	186.349	3970.95	4912.46	20249.93
biconf7bpi	16384	0.06	1.0449	0.0007	0.0023	0.0005	2.01e-07	182.067	3902.02	5605.69	21021.89
biconf7bpi	16384	0.07	1.0498	0.0006	0.0024	0.0005	2.18e-07	183.924	4039.34	6293.70	22070.83
biconf7bpi	16384	0.08	1.0496	0.0005	0.0017	0.0005	2.08e-07	164.284	3093.38	6916.30	20739.55
biconf7bpi	16384	0.09	1.0501	0.0004	0.0019	0.0005	2.35e-07	170.264	3533.36	7509.32	21704.25
option7bpi	1024	0.01	1.2048	0.0868	0.0003	0.0045	7.37e-06	73.7201	628.414	99.6710	594.348
option7bpi	1024	0.02	1.1708	0.0455	0.0002	0.0067	2.12e-05	112.204	1051.01	169.569	955.574
option7bpi	1024	0.03	1.1825	0.0299	0.0003	0.0136	6.22e-05	175.306	1344.40	235.351	1188.02
option7bpi	1024	0.04	1.1525	0.0241	0.0003	0.0119	5.01e-05	159.634	1924.50	285.913	1484.13
option7bpi	1024	0.05	1.1729	0.0195	0.0002	0.0175	0.0001	213.947	2263.92	343.942	1683.57
option7bpi	1024	0.06	1.1880	0.0160	0.0003	0.0233	0.0002	266.659	2449.38	398.302	1811.78
option7bpi	1024	0.07	1.1273	0.0142	0.0002	0.0148	7.49e-05	168.599	2979.04	422.403	1999.58
option7bpi	1024	0.08	1.1462	0.0135	0.0003	0.0204	0.0002	210.847	3736.67	472.015	2297.61
option7bpi	1024	0.09	1.1634	0.0126	0.0002	0.0234	0.0002	254.161	4265.20	519.953	2511.56
option7bpi	2048	0.01	1.1476	0.0458	0.0003	0.0040	7.69e-06	125.948	1374.30	189.872	1256.53
option7bpi	2048	0.02	1.1310	0.0245	0.0004	0.0055	1.44e-05	184.241	2641.40	327.582	2058.78
option7bpi	2048	0.03	1.1596	0.0164	0.0003	0.0128	3.46e-05	311.401	3467.61	461.589	2605.60
option7bpi	2048	0.04	1.1231	0.0132	0.0002	0.0105	4.94e-05	252.373	5255.92	557.247	3259.19

Table A.2: Second Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
option7bpi	2048	0.05	1.1499	0.0114	0.0003	0.0162	6.92e-05	359.098	6732.66	674.411	3927.46
option7bpi	2048	0.06	1.1708	0.0095	0.0003	0.0217	7.47e-05	464.902	7417.36	785.076	4301.61
option7bpi	2048	0.07	1.0996	0.0078	0.0001	0.0049	1.22e-05	246.137	7524.45	824.037	4376.95
option7bpi	2048	0.08	1.1188	0.0079	0.0003	0.0131	8.83e-05	323.157	10432.93	921.477	5365.54
option7bpi	2048	0.09	1.1395	0.0077	0.0002	0.0168	0.0001	406.564	12862.35	1018.50	6199.33
option7bpi	4096	0.01	1.1147	0.0246	0.0002	0.0029	1.93e-06	208.288	3425.38	368.878	2701.11
option7bpi	4096	0.02	1.1051	0.0132	0.0002	0.0038	1.14e-05	292.157	6966.13	640.228	4425.26
option7bpi	4096	0.03	1.1443	0.0093	0.0002	0.0142	1.68e-05	543.518	10099.66	911.064	5904.76
option7bpi	4096	0.04	1.1001	0.0075	0.0002	0.0054	2.10e-05	388.501	14450.08	1091.70	7382.38
option7bpi	4096	0.05	1.1312	0.0069	0.0002	0.0116	7.88e-05	595.061	20592.87	1326.97	9474.83
option7bpi	4096	0.06	1.1568	0.0060	0.0002	0.0192	9.75e-05	805.857	23600.92	1551.34	10892.30
option7bpi	4096	0.07	1.0775	0.0041	0.0002	0.0036	1.86e-05	347.000	17376.28	1614.99	9198.42
option7bpi	4096	0.08	1.0974	0.0046	0.0003	0.0089	7.00e-05	484.478	27749.02	1807.67	12474.38
option7bpi	4096	0.09	1.1186	0.0050	0.0003	0.0159	0.0001	640.708	38058.58	1999.63	15870.10
option7bpi	8192	0.01	1.0915	0.0130	0.0003	0.0022	2.44e-06	330.723	8981.97	722.400	5704.21
option7bpi	8192	0.02	1.0854	0.0071	0.0003	0.0034	7.73e-06	450.710	18775.65	1257.58	9592.87
option7bpi	8192	0.03	1.1324	0.0056	0.0003	0.0113	2.56e-05	943.431	31782.09	1803.12	14352.23
option7bpi	8192	0.04	1.0820	0.0042	0.0002	0.0036	7.89e-06	584.461	38461.46	2147.61	16553.53
option7bpi	8192	0.05	1.1152	0.0044	0.0002	0.0089	5.20e-05	973.014	64057.10	2616.42	24294.06
option7bpi	8192	0.06	1.1432	0.0042	0.0002	0.0181	8.19e-05	1386.22	79191.72	3066.46	30332.23
option7bpi	8192	0.07	1.0615	0.0021	0.0002	0.0036	9.03e-06	473.590	37016.36	3181.97	18775.93
option7bpi	8192	0.08	1.0798	0.0027	0.0002	0.0054	2.88e-05	708.055	70827.22	3557.40	29054.54

Table A.2: Second Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
option7bpi	8192	0.09	1.1002	0.0032	0.0002	0.0101	4.83e-05	987.739	108494.59	3933.69	40756.39
option7bpi	10000	0.01	1.0840	0.0107	0.0003	0.0021	1.57e-06	364.897	11898.53	875.738	6971.07
option7bpi	10000	0.02	1.0803	0.0060	0.0002	0.0025	4.66e-06	500.265	24932.64	1528.00	11972.71
option7bpi	10000	0.03	1.1227	0.0052	0.0002	0.0080	1.47e-05	1067.96	50651.23	2182.29	19664.06
option7bpi	10000	0.04	1.0752	0.0035	0.0004	0.0039	1.12e-05	637.835	49491.54	2605.08	20586.72
option7bpi	10000	0.05	1.1053	0.0039	0.0003	0.0084	3.02e-05	1077.18	92326.46	3165.35	32467.71
option7bpi	10000	0.06	1.1296	0.0042	0.0002	0.0124	8.20e-05	1545.85	129186.64	3698.67	45204.48
option7bpi	10000	0.07	1.0577	0.0017	0.0002	0.0030	8.33e-06	508.625	44429.39	3870.34	22726.69
option7bpi	10000	0.08	1.0733	0.0022	0.0002	0.0051	2.16e-05	763.676	87829.81	4316.54	35319.44
option7bpi	10000	0.09	1.0916	0.0028	0.0002	0.0102	5.35e-05	1077.18	142018.05	4764.52	52567.23
option7bpi	16384	0.01	1.0745	0.0069	0.0003	0.0012	8.31e-07	511.072	23935.50	1422.32	12177.64
option7bpi	16384	0.02	1.0702	0.0039	0.0002	0.0019	3.45e-06	679.314	49769.67	2480.00	20967.27
option7bpi	16384	0.03	1.1210	0.0036	0.0002	0.0089	1.43e-05	1623.23	103597.50	3570.19	37119.56
option7bpi	16384	0.04	1.0663	0.0023	0.0001	0.0030	6.17e-06	851.096	95082.59	4233.00	36722.19
option7bpi	16384	0.05	1.1003	0.0029	0.0003	0.0078	2.83e-05	1567.33	195401.92	5162.71	64593.47
option7bpi	16384	0.06	1.1312	0.0031	0.0002	0.0157	4.30e-05	2371.44	264464.26	6068.55	89040.04
option7bpi	16384	0.07	1.0501	0.0010	0.0002	0.0025	4.69e-06	625.383	71673.68	6295.53	37224.79
option7bpi	16384	0.08	1.0652	0.0015	0.0002	0.0036	8.18e-06	998.162	163051.13	7018.57	63528.56
option7bpi	16384	0.09	1.0841	0.0020	0.0002	0.0066	3.77e-05	1482.16	290039.82	7752.72	102570.19
option8bpi	1024	0.01	1.2135	0.0813	0.0003	0.0038	5.24e-06	72.8116	588.240	100.385	556.506
option8bpi	1024	0.02	1.1765	0.0415	0.0002	0.0063	1.98e-05	114.504	938.867	170.385	871.100
option8bpi	1024	0.03	1.1640	0.0269	0.0002	0.0060	2.31e-05	134.467	1101.12	231.694	1067.92

Table A.2: Second Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
option8bpi	1024	0.04	1.1657	0.0197	0.0001	0.0147	0.0001	186.509	1313.60	289.198	1211.71
option8bpi	1024	0.05	1.1582	0.0167	0.0002	0.0143	0.0001	170.514	1480.79	339.659	1434.32
option8bpi	1024	0.06	1.1671	0.0137	5.00e-05	0.0266	3.45e-05	215.129	1636.59	391.308	1545.24
option8bpi	1024	0.07	1.1757	0.0115	0.0001	0.0282	0.0002	259.262	1766.73	440.518	1618.94
option8bpi	1024	0.08	1.1830	0.0099	6.00e-05	0.0495	8.48e-06	302.286	1870.03	487.180	1677.48
option8bpi	1024	0.09	1.1741	0.0088	0.0002	0.0208	0.0003	242.443	1814.18	524.720	1755.66
option8bpi	2048	0.01	1.1526	0.0422	1.00e-04	0.0041	8.74e-06	128.415	1241.46	190.707	1156.27
option8bpi	2048	0.02	1.1410	0.0217	0.0002	0.0061	2.29e-05	210.263	1995.88	330.489	1822.57
option8bpi	2048	0.03	1.1424	0.0144	0.0002	0.0115	2.60e-05	252.764	2387.55	454.766	2288.37
option8bpi	2048	0.04	1.1541	0.0104	5.00e-05	0.0139	0.0001	359.586	2802.12	572.697	2561.53
option8bpi	2048	0.05	1.1411	0.0092	9.00e-05	0.0112	7.53e-05	308.102	3768.33	669.269	3183.46
option8bpi	2048	0.06	1.1559	0.0076	0.0001	0.0192	0.0002	399.240	4126.21	775.115	3427.30
option8bpi	2048	0.07	1.1685	0.0064	4.00e-05	0.0295	0.0003	489.620	4374.62	875.670	3598.06
option8bpi	2048	0.08	1.1779	0.0055	4.00e-05	0.0308	0.0003	576.942	4585.46	970.181	3764.71
option8bpi	2048	0.09	1.1695	0.0050	7.00e-05	0.0294	5.49e-05	459.792	4596.17	1045.39	4023.64
option8bpi	4096	0.01	1.1235	0.0223	8.00e-05	0.0018	2.38e-06	237.574	2662.96	371.805	2446.09
option8bpi	4096	0.02	1.1240	0.0116	9.00e-05	0.0080	1.12e-05	380.880	4875.56	651.168	3886.02
option8bpi	4096	0.03	1.1327	0.0078	5.00e-05	0.0110	1.02e-05	470.023	5804.97	901.861	4927.38
option8bpi	4096	0.04	1.1496	0.0057	1.00e-04	0.0199	6.33e-05	686.890	6773.91	1140.82	5659.74
option8bpi	4096	0.05	1.1291	0.0053	8.00e-05	0.0062	2.11e-05	546.867	10781.55	1324.50	7267.97
option8bpi	4096	0.06	1.1479	0.0043	6.00e-05	0.0151	5.93e-05	731.913	11574.52	1539.49	7805.92
option8bpi	4096	0.07	1.1625	0.0037	1.00e-04	0.0338	2.35e-05	913.865	12191.17	1742.37	8294.76

Table A.2: Second Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
option8bpi	4096	0.08	1.1737	0.0032	9.00e-05	0.0428	7.42e-06	1089.89	12730.23	1933.44	8777.59
option8bpi	4096	0.09	1.1646	0.0030	5.00e-05	0.0283	3.02e-05	860.659	13031.49	2082.01	9529.10
option8bpi	8192	0.01	1.1081	0.0117	6.00e-05	0.0039	7.52e-06	430.984	6421.57	733.407	5118.96
option8bpi	8192	0.02	1.1132	0.0063	6.00e-05	0.0059	7.52e-06	677.762	13620.87	1289.76	8470.25
option8bpi	8192	0.03	1.1260	0.0043	3.00e-05	0.0076	2.70e-05	862.677	15829.52	1793.12	10895.01
option8bpi	8192	0.04	1.1466	0.0032	7.00e-05	0.0209	3.89e-06	1300.26	17772.52	2275.78	12547.06
option8bpi	8192	0.05	1.1191	0.0032	2.00e-05	0.0131	6.57e-06	963.769	33059.10	2625.57	17600.69
option8bpi	8192	0.06	1.1404	0.0027	8.00e-05	0.0171	7.37e-05	1333.92	36006.24	3058.86	19276.40
option8bpi	8192	0.07	1.1573	0.0023	6.00e-05	0.0243	0.0002	1701.26	37581.33	3469.19	20650.31
option8bpi	8192	0.08	1.1699	0.0021	8.00e-05	0.0407	8.49e-06	2055.61	39530.75	3854.17	22832.40
option8bpi	8192	0.09	1.1588	0.0019	6.00e-05	0.0239	2.51e-05	1600.84	40245.86	4143.19	24559.91
option8bpi	10000	0.01	1.1062	0.0099	8.51e-05	0.0026	3.35e-06	505.519	8794.80	893.698	6450.64
option8bpi	10000	0.02	1.1079	0.0055	6.00e-05	0.0066	1.06e-05	782.416	20473.38	1566.96	11096.39
option8bpi	10000	0.03	1.1211	0.0039	6.00e-05	0.0078	1.75e-05	1007.10	24262.07	2179.37	14831.15
option8bpi	10000	0.04	1.1444	0.0029	9.00e-05	0.0175	3.94e-06	1544.53	27153.42	2772.60	17232.21
option8bpi	10000	0.05	1.1085	0.0031	9.00e-05	0.0088	2.39e-05	1089.81	54539.97	3174.58	25736.76
option8bpi	10000	0.06	1.1247	0.0032	1.00e-04	0.0152	7.44e-05	1509.24	71351.51	3682.83	34059.81
option8bpi	10000	0.07	1.1376	0.0033	7.00e-05	0.0200	0.0002	1927.59	88946.66	4162.79	44344.45
option8bpi	10000	0.08	1.1467	0.0035	0.0001	0.0215	0.0003	2332.06	107151.62	4611.54	56702.58
option8bpi	10000	0.09	1.1375	0.0030	3.00e-05	0.0199	8.85e-05	1792.84	100089.77	4964.83	57270.73
option8bpi	16384	0.01	1.0990	0.0062	5.00e-05	0.0028	2.44e-06	768.451	17501.43	1454.75	10947.31
option8bpi	16384	0.02	1.1052	0.0035	6.00e-05	0.0051	9.15e-06	1200.10	40888.58	2561.21	19090.63

Table A.2: Second Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
option8bpi	16384	0.03	1.1196	0.0025	6.00e-05	0.0098	7.07e-06	1573.55	47953.70	3565.81	25430.86
option8bpi	16384	0.04	1.1428	0.0019	9.00e-05	0.0177	6.12e-05	2451.89	54002.16	4536.49	30746.87
option8bpi	16384	0.05	1.1093	0.0021	4.00e-05	0.0091	1.20e-05	1685.60	107843.63	5205.34	46069.88
option8bpi	16384	0.06	1.1333	0.0018	7.00e-05	0.0170	1.01e-05	2427.57	118712.13	6079.82	52160.26
option8bpi	16384	0.07	1.1521	0.0016	7.00e-05	0.0182	0.0001	3163.15	127291.80	6907.09	58092.13
option8bpi	16384	0.08	1.1657	0.0015	5.00e-05	0.0291	0.0002	3875.16	133664.03	7681.11	65759.91
option8bpi	16384	0.09	1.1528	0.0014	5.00e-05	0.0259	2.34e-06	2972.78	136046.51	8243.33	70396.98
originalbpi	1024	0.01	1.1087	0.0638	0.0433	0.0026	1.92e-06	37.1169	299.382	91.3890	437.092
originalbpi	1024	0.02	1.1154	0.0292	0.0103	0.0022	7.35e-07	43.0996	296.698	161.495	612.943
originalbpi	1024	0.03	1.1336	0.0182	0.0051	0.0020	2.29e-07	44.4332	271.464	225.619	721.674
originalbpi	1024	0.04	1.1342	0.0128	0.0032	0.0020	1.39e-07	45.7369	256.119	281.376	786.997
originalbpi	1024	0.05	1.1448	0.0093	0.0019	0.0020	4.04e-08	43.2476	216.339	335.734	803.971
originalbpi	1024	0.06	1.1450	0.0077	0.0014	0.0020	8.06e-08	44.8787	217.487	383.899	863.121
originalbpi	1024	0.07	1.1593	0.0062	0.0010	0.0020	0.00e+00	43.3512	186.831	434.381	870.140
originalbpi	1024	0.08	1.1559	0.0053	0.0007	0.0020	0.00e+00	44.6810	191.956	476.022	897.730
originalbpi	1024	0.09	1.1544	0.0044	0.0006	0.0020	0.00e+00	44.3931	179.551	515.936	884.892
originalbpi	2048	0.01	1.1150	0.0312	0.0106	0.0011	1.46e-07	49.7236	403.903	184.446	853.483
originalbpi	2048	0.02	1.1183	0.0143	0.0026	0.0010	5.64e-08	53.2146	351.817	323.911	1201.78
originalbpi	2048	0.03	1.1314	0.0090	0.0014	0.0010	0.00e+00	54.0839	327.117	450.413	1425.19
originalbpi	2048	0.04	1.1368	0.0062	0.0007	0.0010	0.00e+00	54.2866	289.143	564.087	1538.04
originalbpi	2048	0.05	1.1440	0.0046	0.0004	0.0010	0.00e+00	50.8016	237.128	670.976	1591.54
originalbpi	2048	0.06	1.1469	0.0038	0.0003	0.0010	0.00e+00	52.5904	241.055	769.126	1704.36

Table A.2: Second Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
originalbpi	2048	0.07	1.1601	0.0031	0.0003	0.0010	0.00e+00	50.5106	205.055	869.372	1733.51
originalbpi	2048	0.08	1.1565	0.0026	0.0003	0.0010	0.00e+00	51.8366	211.105	952.550	1787.05
originalbpi	2048	0.09	1.1549	0.0022	0.0002	0.0010	5.59e-08	51.2062	195.146	1032.34	1759.44
originalbpi	4096	0.01	1.1134	0.0156	0.0026	0.0005	7.08e-09	61.8556	492.770	368.456	1704.99
originalbpi	4096	0.02	1.1211	0.0071	0.0008	0.0005	2.84e-09	63.8099	409.433	649.475	2390.89
originalbpi	4096	0.03	1.1313	0.0044	0.0004	0.0005	0.00e+00	63.3996	356.439	900.789	2808.83
originalbpi	4096	0.04	1.1375	0.0031	0.0001	0.0005	0.00e+00	62.8900	312.652	1128.86	3101.52
originalbpi	4096	0.05	1.1433	0.0023	1.00e-04	0.0005	0.00e+00	58.5491	253.611	1341.14	3148.87
originalbpi	4096	0.06	1.1481	0.0019	6.00e-05	0.0005	0.00e+00	60.3983	254.458	1539.84	3358.96
originalbpi	4096	0.07	1.1593	0.0015	4.00e-05	0.0005	0.00e+00	57.5164	218.809	1737.60	3407.69
originalbpi	4096	0.08	1.1567	0.0013	7.00e-05	0.0005	0.00e+00	59.0728	223.553	1905.49	3547.85
originalbpi	4096	0.09	1.1556	0.0011	5.00e-05	0.0005	0.00e+00	58.0084	206.839	2065.98	3506.84
originalbpi	8192	0.01	1.1103	0.0077	0.0006	0.0002	9.61e-10	73.7413	551.688	734.879	3364.94
originalbpi	8192	0.02	1.1219	0.0035	0.0002	0.0002	0.00e+00	74.2498	444.117	1299.94	4744.70
originalbpi	8192	0.03	1.1321	0.0022	0.0001	0.0002	0.00e+00	72.7182	373.794	1802.75	5609.98
originalbpi	8192	0.04	1.1389	0.0016	6.00e-05	0.0002	0.00e+00	71.7671	329.711	2260.52	6147.91
originalbpi	8192	0.05	1.1431	0.0011	1.00e-05	0.0002	0.00e+00	66.4632	267.502	2681.88	6300.55
originalbpi	8192	0.06	1.1481	0.0009	1.00e-05	0.0002	0.00e+00	68.2637	271.069	3079.77	6753.60
originalbpi	8192	0.07	1.1594	0.0008	0.00e+00	0.00e+00	0.00e+00	64.5331	223.041	3475.33	6862.89
originalbpi	8192	0.08	1.1569	0.0006	0.00e+00	0.00e+00	0.00e+00	66.2150	232.382	3811.45	7051.90
originalbpi	8192	0.09	1.1559	0.0005	2.00e-05	0.0002	0.00e+00	64.8823	215.035	4133.02	6982.26
originalbpi	10000	0.01	1.1088	0.0062	0.0004	0.0002	9.99e-10	77.2815	565.182	895.810	4057.12

Table A.2: Second Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
originalbpi	10000	0.02	1.1226	0.0029	0.0001	0.0002	0.00e+00	77.2029	448.453	1587.78	5760.57
originalbpi	10000	0.03	1.1321	0.0018	3.00e-05	0.0002	0.00e+00	75.5511	384.917	2200.76	6790.45
originalbpi	10000	0.04	1.1387	0.0013	1.00e-05	0.0002	0.00e+00	74.2211	333.840	2759.06	7523.47
originalbpi	10000	0.05	1.1428	0.0009	1.00e-05	0.0002	0.00e+00	68.6657	268.967	3273.01	7720.09
originalbpi	10000	0.06	1.1482	0.0008	3.00e-05	0.0002	0.00e+00	70.5289	275.638	3759.82	8277.98
originalbpi	10000	0.07	1.1595	0.0006	0.00e+00	0.00e+00	0.00e+00	66.7339	228.493	4242.82	8376.96
originalbpi	10000	0.08	1.1564	0.0005	0.00e+00	0.00e+00	0.00e+00	68.2108	237.322	4650.87	8581.32
originalbpi	10000	0.09	1.1557	0.0004	1.00e-05	0.0002	0.00e+00	66.8180	215.206	5044.29	8503.44
originalbpi	16384	0.01	1.1092	0.0038	0.0002	0.0001	0.00e+00	85.9333	607.062	1468.26	6684.14
originalbpi	16384	0.02	1.1226	0.0018	7.00e-05	0.0001	0.00e+00	84.5903	461.677	2601.36	9399.07
originalbpi	16384	0.03	1.1325	0.0011	5.00e-05	0.0001	0.00e+00	82.2690	397.823	3606.96	11166.39
originalbpi	16384	0.04	1.1389	0.0008	2.00e-05	0.0001	0.00e+00	80.5696	348.102	4520.96	12304.97
originalbpi	16384	0.05	1.1429	0.0006	0.00e+00	0.00e+00	0.00e+00	74.1179	273.818	5362.67	12611.13
originalbpi	16384	0.06	1.1482	0.0005	0.00e+00	0.00e+00	0.00e+00	76.1496	282.565	6160.02	13598.51
originalbpi	16384	0.07	1.1594	0.0004	1.00e-05	0.0001	0.00e+00	71.7353	233.357	6950.88	13784.74
originalbpi	16384	0.08	1.1569	0.0003	0.00e+00	0.00e+00	0.00e+00	73.4901	241.640	7623.23	14147.33
originalbpi	16384	0.09	1.1560	0.0003	0.00e+00	0.00e+00	0.00e+00	71.8769	221.918	8266.81	13957.82
yanetalbpi	1024	0.01	1.1503	0.0806	0.0032	0.0039	5.12e-06	56.0206	524.584	95.0974	552.547
yanetalbpi	1024	0.02	1.1314	0.0414	0.0031	0.0048	1.17e-05	79.2855	855.935	163.782	870.579
yanetalbpi	1024	0.03	1.1124	0.0279	0.0033	0.0063	2.15e-05	93.0032	1157.81	221.309	1110.18
yanetalbpi	1024	0.04	1.1122	0.0206	0.0030	0.0071	2.63e-05	100.771	1369.72	275.823	1273.06
yanetalbpi	1024	0.05	1.0990	0.0168	0.0032	0.0068	2.99e-05	108.055	1638.62	322.180	1449.77

Table A.2: Second Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
yanetalbpi	1024	0.06	1.1047	0.0143	0.0032	0.0087	4.71e-05	119.425	1950.72	370.234	1618.39
yanetalbpi	1024	0.07	1.1073	0.0123	0.0034	0.0096	5.98e-05	124.355	2168.29	414.743	1738.87
yanetalbpi	1024	0.08	1.0991	0.0101	0.0031	0.0080	5.03e-05	115.545	2001.31	452.533	1713.85
yanetalbpi	1024	0.09	1.0963	0.0091	0.0033	0.0084	5.85e-05	121.347	2246.10	489.832	1828.84
yanetalbpi	2048	0.01	1.1135	0.0421	0.0034	0.0027	3.32e-06	89.6232	1122.89	184.120	1156.35
yanetalbpi	2048	0.02	1.0973	0.0213	0.0032	0.0034	7.39e-06	118.064	1868.99	317.716	1798.02
yanetalbpi	2048	0.03	1.0883	0.0144	0.0034	0.0041	1.06e-05	135.933	2555.03	433.069	2287.15
yanetalbpi	2048	0.04	1.0867	0.0105	0.0033	0.0043	1.36e-05	142.853	2958.72	539.047	2590.83
yanetalbpi	2048	0.05	1.0754	0.0084	0.0034	0.0041	1.24e-05	148.716	3374.01	630.595	2884.60
yanetalbpi	2048	0.06	1.0818	0.0072	0.0033	0.0053	2.08e-05	166.257	4149.39	725.236	3256.16
yanetalbpi	2048	0.07	1.0839	0.0062	0.0033	0.0058	2.76e-05	171.409	4571.88	812.088	3505.84
yanetalbpi	2048	0.08	1.0796	0.0051	0.0033	0.0050	2.00e-05	158.711	4185.47	889.057	3498.09
yanetalbpi	2048	0.09	1.0764	0.0045	0.0033	0.0052	2.55e-05	160.900	4394.96	962.012	3579.73
yanetalbpi	4096	0.01	1.0858	0.0218	0.0033	0.0018	1.76e-06	134.556	2422.83	359.151	2395.64
yanetalbpi	4096	0.02	1.0749	0.0109	0.0031	0.0021	3.25e-06	168.329	4019.77	622.553	3674.27
yanetalbpi	4096	0.03	1.0683	0.0072	0.0035	0.0023	4.31e-06	187.343	5261.08	850.398	4555.91
yanetalbpi	4096	0.04	1.0696	0.0054	0.0035	0.0026	6.03e-06	197.282	6199.58	1061.26	5336.47
yanetalbpi	4096	0.05	1.0592	0.0041	0.0037	0.0024	5.23e-06	198.087	6525.62	1242.27	5634.82
yanetalbpi	4096	0.06	1.0656	0.0036	0.0030	0.0030	9.13e-06	225.273	8462.56	1428.87	6474.71
yanetalbpi	4096	0.07	1.0670	0.0030	0.0032	0.0036	1.24e-05	226.594	8846.74	1598.99	6816.62
yanetalbpi	4096	0.08	1.0652	0.0025	0.0029	0.0028	8.14e-06	204.132	7537.38	1754.57	6696.94
yanetalbpi	4096	0.09	1.0635	0.0022	0.0032	0.0028	9.73e-06	210.090	8083.20	1901.11	6954.84

Table A.2: Second Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
yanetalbpi	8192	0.01	1.0658	0.0110	0.0034	0.0010	7.54e-07	192.408	5148.37	705.183	4833.97
yanetalbpi	8192	0.02	1.0606	0.0055	0.0033	0.0012	1.43e-06	233.205	8246.17	1228.57	7349.82
yanetalbpi	8192	0.03	1.0553	0.0035	0.0039	0.0015	2.31e-06	252.121	10126.08	1680.18	8963.63
yanetalbpi	8192	0.04	1.0575	0.0026	0.0031	0.0016	2.74e-06	256.046	10992.94	2098.75	10098.55
yanetalbpi	8192	0.05	1.0478	0.0020	0.0031	0.0015	2.43e-06	254.713	11423.91	2458.01	10814.73
yanetalbpi	8192	0.06	1.0547	0.0018	0.0032	0.0020	3.56e-06	295.483	15535.34	2828.77	12628.52
yanetalbpi	8192	0.07	1.0561	0.0015	0.0036	0.0019	3.80e-06	292.964	15737.73	3165.39	13124.73
yanetalbpi	8192	0.08	1.0556	0.0011	0.0032	0.0014	2.51e-06	253.527	11997.20	3477.52	12288.20
yanetalbpi	8192	0.09	1.0533	0.0010	0.0031	0.0016	3.76e-06	260.331	12899.26	3765.82	12793.21
yanetalbpi	10000	0.01	1.0632	0.0092	0.0030	0.0010	6.93e-07	216.995	6725.48	858.746	6018.54
yanetalbpi	10000	0.02	1.0577	0.0045	0.0035	0.0011	1.12e-06	253.318	9998.97	1495.71	8951.40
yanetalbpi	10000	0.03	1.0531	0.0029	0.0030	0.0013	1.94e-06	272.852	12171.82	2046.76	10856.90
yanetalbpi	10000	0.04	1.0544	0.0021	0.0029	0.0012	1.50e-06	274.305	12920.19	2554.52	12199.01
yanetalbpi	10000	0.05	1.0456	0.0016	0.0034	0.0013	1.78e-06	274.572	13367.96	2994.20	13124.94
yanetalbpi	10000	0.06	1.0517	0.0014	0.0034	0.0015	2.66e-06	316.097	18267.16	3443.21	15222.72
yanetalbpi	10000	0.07	1.0541	0.0012	0.0034	0.0016	3.27e-06	314.402	18390.35	3856.81	15818.06
yanetalbpi	10000	0.08	1.0534	0.0009	0.0034	0.0015	2.57e-06	269.985	13813.55	4236.27	14766.38
yanetalbpi	10000	0.09	1.0510	0.0008	0.0030	0.0013	2.24e-06	274.551	14202.47	4587.11	15054.52
yanetalbpi	16384	0.01	1.0533	0.0055	0.0035	0.0007	4.29e-07	266.679	10536.81	1393.90	9673.65
yanetalbpi	16384	0.02	1.0502	0.0026	0.0033	0.0009	7.74e-07	303.350	14764.76	2433.37	14144.60
yanetalbpi	16384	0.03	1.0456	0.0017	0.0036	0.0008	7.68e-07	322.005	17415.91	3329.83	17107.06
yanetalbpi	16384	0.04	1.0486	0.0012	0.0033	0.0008	8.50e-07	318.248	17335.47	4162.19	18987.35

Table A.2: Second Experiment Results

algorithm	key len	error rate	avg eff	var eff	fer	avg ber	var ber	avg cu	var cu	avg msg len	var msg len
yanetalbpi	16384	0.05	1.0400	0.0009	0.0035	0.0009	9.30e-07	317.120	17741.32	4879.68	20113.81
yanetalbpi	16384	0.06	1.0463	0.0008	0.0035	0.0010	1.12e-06	368.501	25122.75	5612.90	23551.70
yanetalbpi	16384	0.07	1.0491	0.0007	0.0034	0.0011	1.44e-06	365.996	25191.83	6289.30	24300.54
yanetalbpi	16384	0.08	1.0492	0.0005	0.0032	0.0008	7.56e-07	306.399	17172.32	6913.28	22556.97
yanetalbpi	16384	0.09	1.0470	0.0005	0.0034	0.0009	1.18e-06	313.405	18506.83	7486.60	23215.18